This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

COSMIC: Leader-Driven Context-Oriented Collaboration Between Agents

Vansh Vashishtha, Sonal Pandey, Yash Raj Singh, Arshdeep Bahga, and Vijay K. Madisetti, *Fellow, IEEE*

***Abstract*—We present COSMIC, a leader-driven multiagent coordination framework that extends the AutoGen GroupChat paradigm to support more stable and context-consistent collaboration among large language model (LLM) agents. COSMIC introduces: 1) a leader agent that decomposes tasks and issues an explicit termination decision; 2) a shared rolling memory buffer that synchronizes recent context across agents; and 3) a speaker selector agent (SSA) that enforces structured turn-taking and mitigates self-invocation and speaker imbalance. Across six coordination-intensive tasks, COSMIC achieves 100% validity under task-specific hard-constraint criteria. On the five tasks where baseline frameworks produced validated outputs, COSMIC improves composite scores by 20.2%, 23.3%, 27.7%, 52.6%, and 46.9% over CrewAI Hierarchical, CrewAI Sequential, AutoGen, LangChain, and OpenAI Swarm, respectively. COSMIC reduces wall-clock time relative to CrewAI Hierarchical, CrewAI Sequential, AutoGen, and LangChain by 57.3%, 50.8%, 61.4%, and 77.6%, while incurring higher runtime than OpenAI Swarm due to richer coordination and memory overhead. On GSM8K, COSMIC attains $94 \pm 1\%$ accuracy, exceeding the sparse multiagent debate (MAD) configuration reported in a Google Research publication. Empirically, performance peaks around five agents, reflecting a trade-off between diversity and coordination overhead.**

***Index Terms*—Agent communication, AutoGen, collaborative AI, leader-driven termination, multiagent coordination, shared memory.**

I. Introduction

DESIGNING multiagent systems with large language models (LLMs) presents significant coordination challenges. When multiple autonomous LLM agents are naively chained together, they often exhibit inconsistent reasoning, fragmented context, and cascading hallucinations [1], [2]. Ensuring coherent, reliable collaboration therefore requires explicit design of communication protocols, shared state, and roles. At the same time, recent work shows that LLM-based collectives can solve tasks that are difficult for a single agent, including complex reasoning, planning, and tool use [3], [4].

Existing multiagent frameworks typically adopt one of three collaboration patterns. Sequential pipelines pass intermediate results along a fixed chain of agents, which simplifies control but makes it hard to revisit or jointly refine earlier decisions. Hierarchical manager–worker frameworks delegate subtasks from a top-level planner to worker agents, improving reliability but restricting rich peer-to-peer negotiation [5]. More recent group-chat frameworks such as AutoGen create a shared chatroom where multiple agents converse freely. In practice, each of these patterns suffers from concrete shortcomings that matter in high-dependency tasks: unstable turn-taking and termination, fragmented shared context, and weak global oversight.

In group-chat settings, agents communicate via free-form natural language, which encourages flexibility but also leads to over-communication, redundant messages, and difficulty deciding when to stop. Speaker-selection heuristics can produce unbalanced participation or self-invocation loops in which one agent dominates the conversation. Similarly, many frameworks maintain only local or per-agent histories, so no component has a consistent global view of the evolving discussion; this causes forgotten constraints, contradictory proposals, or repeated questions [13], [20]. Finally, while hierarchical systems introduce a manager agent, its behavior is often scripted via fixed workflows rather than a dynamic leader that actively monitors convergence and decides when a solution is "good enough."

This article addresses three concrete challenges in LLM-based multiagent collaboration.

1) *Robust coordination and termination:* How can we design a leader that guides the discussion and provides a reliable, explicit termination signal, while still allowing rich back-and-forth communication and coordination among agents?
2) *Consistent shared context:* How can we maintain a global, rolling memory that keeps all agents synchronized without exceeding context and latency budgets?
3) *Fair, task-aware turn-taking:* How can we prevent self-invocation loops and under-utilized agents by structuring who speaks next, without hard-coding a rigid conversation order?

To tackle these challenges, we introduce COSMIC, a leader-driven, context-oriented multiagent coordination framework that extends the AutoGen group-chat paradigm. COSMIC

Received 16 May 2025; revised 21 November 2025 and 20 January 2026; accepted 27 January 2026. *(Corresponding author: Vijay K. Madisetti.)*

Vansh Vashishtha, Sonal Pandey, and Yash Raj Singh are with Bennett University, Greater Noida, Uttar Pradesh 201310, India (e-mail: victoriousvansh@gmail.com; pandey2sonal@gmail.com; yashraj.singh7158work@gmail.com).

Arshdeep Bahga is with the Cloudemy Technology Labs, Chandigarh 160047, India (e-mail: arshdeep@vptinc.net).

Vijay K. Madisetti is with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA 30332, USA (e-mail: vkm@gatech.edu).

Digital Object Identifier 10.1109/TCSS.2026.3661839

© 2026 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License. For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

augments group chat with three key components: 1) a leader agent that plans and terminates multiagent sessions; 2) a shared rolling memory buffer that exposes the last N messages to all agents; and 3) a speaker selector agent (SSA) that autonomously decides which agent should act next given the global context. Taken together, these components transform loosely managed multiagent chat into a structured yet flexible collaboration protocol for high-coupling, high-cohesion tasks.

Our contributions are as follows.

1) *Leader-guided collaboration:* We formalize a leader-driven coordination mechanism in which a dedicated Leader Agent initializes tasks, monitors progress in the shared memory, and posts an explicit termination signal once a satisfactory solution has emerged.
2) *Shared rolling memory:* We design a rolling shared-memory buffer visible to all agents, analyze its utilization and depth trade-offs, and show empirically that moderate window sizes yield strong performance on multistep reasoning and coordination tasks.
3) *SSA:* We introduce an SSA that observes the full conversation context, enforces anti-self-invocation constraints, and selects the next speaker based on recent participation and task needs, significantly reducing loops and imbalanced dialogues.
4) *Empirical evaluation across six tasks:* Across six collaboration-intensive tasks: 1) mathematical reasoning; 2) multilingual translation; 3) story generation; 4) healthcare diagnostics; 5) bandwidth allocation; and 6) resource scheduling—COSMIC consistently outperforms popular baselines (CrewAI hierarchical and sequential, AutoGen, LangChain, and OpenAI Swarm) in both solution quality and execution time, and attains 94 ± 1 accuracy on GSM8K, surpassing a published multiagent debate (MAD) configuration by Google. Moreover, we empirically identify that teams of around five agents offer a favorable trade-off between diversity and communication overhead.

The remainder of this article is organized as follows. Section II reviews related work and identifies key research gaps in multiagent LLM collaboration. Section III presents the COSMIC framework, including the leader agent, shared memory buffer, and the SSA. Section IV defines the problem formulation and task categorization criteria based on coupling and cohesion. Section V analyzes the shared memory model and its utilization and context-depth trade-offs. Section VI describes the tasks, metrics, and experimental setup used for evaluation. Section VII analyzes the optimal team size for collaboration. Section VIII reports quantitative benchmarking results and comparisons to recent frameworks. Section IX presents a warehouse coordination setting as a stress test of pure collaboration. Section X discusses limitations and future directions, and Section XI concludes.

II. Related Work and Research Gaps

A. Coordination Structures in Multiagent LLM Systems

Traditional multiagent systems often rely on constrained message passing and limited state sharing (e.g., exchanging only specific state variables or signals) to coordinate agents [3]. By contrast, LLM-based agents usually communicate in free-form natural language, which greatly increases flexibility but can also introduce inefficiencies and ambiguity. Without structured protocols, agents tend to over-communicate, producing redundant or confusing messages [3]. Prompt-based communication policies and role-specific channels have been proposed to mitigate these issues by constraining the interaction surface while still leveraging natural language [3], [8]. Establishing well-defined collaboration channels is therefore seen as crucial for successful multiagent LLM deployments [8].

Many frameworks organize multiple LLM agents in *sequential* or *hierarchical* structures to break down complex tasks. In a sequential pipeline, agents perform subtasks one after another, passing along intermediate results. Hierarchical setups designate a top-level planner or manager agent that delegates subtasks to worker agents. Such configurations have been shown to improve reliability and resilience: for instance, Huang et al. [9] compared a fully sequential chain (Agent A $\rightarrow$ B $\rightarrow$ C), a fully decentralized group (all agents conversing freely), and a hierarchical setup where one agent (A) oversees the interaction of others (B $\leftrightarrow$ C). The hierarchical structure suffered the smallest performance drop (only $\sim 9.2\%$) when facing faulty agents, whereas the purely sequential and fully decentralized structures degraded much more (26.0% and 31.2%, respectively) [9], [21]. The CAMEL framework (Li et al., 2023) exemplifies a simple hierarchy: a "manager" agent role-plays instructions to a "worker" agent to solve problems collaboratively [10]. More recent systems such as Captain Agent dynamically assemble and instruct a team of agents for each problem step [11]. Platforms such as CrewAI similarly adopt leader–worker patterns to route subtasks to appropriate specialists [8].

These approaches clearly demonstrate the benefits of leader-driven coordination for robustness and reliability. However, they often rely on predetermined workflows or manually scripted coordination logic. As a result, complex tasks such as healthcare diagnosis, warehouse coordination, or collaborative story writing where requirements can change midconversation and multiple skills must interact may be constrained by rigid pipelines and limited peer-to-peer negotiation. In many cases, there is no *adaptive* leader agent that continuously monitors progress, updates task decomposition, and decides when a solution is "good enough." This overreliance on fixed hierarchical or sequential pipelines leaves a gap for frameworks that combine the resilience of leader-based structures with the flexibility of free-form group discussion.

B. Turn-Taking and Termination in Group-Chat Frameworks

Emerging LLM frameworks support *group-chat* interactions among multiple agents. AutoGen [7], for example, creates a shared chatroom where agents converse to solve a task. A GroupChatManager component implements predetermined speaker-switching rules to decide which agent should speak next and broadcasts messages to the group [7]. Nonomura and Mori [12] show that such mechanisms can significantly affect conversation quality: automatic speaker selection

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

reduces dialogue breakdowns and improves logical coherence in multi-LLM discussions.

Despite these advances, group-chat frameworks still face challenges. Without careful design, multi-agent LLM dialogues can become unstructured, agents may ramble, repeat information, or enter infinite loops. Unlike human conversations, which rely on subtle turn-taking cues and implicit stopping signals, LLM agents in text-based chats lack natural interaction management [12]. A known issue is the absence of robust termination mechanisms: agents may not know when the task is complete or when additional messages cease to add value. Existing systems typically impose hard limits on the number of turns or rely on simple heuristic triggers to end the conversation. Wu et al. [7] explicitly measure "termination failures," where a conversation fails to stop on its own, highlighting the need for better stop criteria.

Turn-taking policy is a central factor here. Most existing systems either rely on simple round-robin schemes or permit each agent to nominate the subsequent speaker without any external oversight [1]. Such approaches can lead to self-invocation loops, where one agent repeatedly selects itself, or to instances where certain agents rarely receive a turn. OpenAI Swarm and related frameworks aim to introduce more flexible routing, but still depend heavily on each agent's internal logic for speaker selection, without a dedicated arbiter to enforce fairness or prevent deadlock [6]. Nonomura and Mori [12], [23] show empirically that relying solely on local agent heuristics can lead to deadlocks or one agent dominating the conversation.

In summary, while group-chat frameworks enable rich, multidirectional interaction, they often lack robust, externally governed mechanisms for both turn-taking and termination. This motivates designs with a specialized SSA or equivalent component that can act as an impartial arbiter to prevent self-invocation loops, ensure balanced participation, and coordinate with a termination policy rooted in task completion rather than raw turn counts.

C. Shared Memory and State Sharing

Another limitation of many multiagent LLM systems is the absence of a centralized, persistent memory that all agents can draw upon. Typically, each agent maintains only a minimal local context (e.g., the last few turns it has seen), which leads to fragmentation of knowledge and inconsistent views of the global state. Over long or complex collaborations, important context may be forgotten or siloed within a single agent's history. This can yield repetitive questions, contradictions, or inconsistent decisions across agents.

To address this, Gao and Zhang [13] propose a memory sharing framework that maintains a real-time shared memory store accessible to all agents. In their approach, each agent's interactions (queries and results) are logged into a common memory pool that agents can query to recall relevant past information [13]. They show that enabling agents to log and retrieve information from a common memory significantly improves task success rates, especially on open-ended problems that require referring back to earlier context [13], [22]. In the absence of shared memory, agents in their experiments tended to repeat questions or contradict each other due to lack of a unified knowledge state.

Other work explores using vector databases or knowledge graphs as shared memory to support retrieval of long-term context in multi-agent settings [15]. However, many frameworks still maintain only a minimal global context, often just the most recent dialogue turn or a small context window [14], [15]. Distributed memory solutions may not guarantee that every agent possesses an up-to-date, consistent view of the conversation. As a result, maintaining *continuous, shared state* that is both accessible and aligned across agents remains an open challenge. Ablation-style comparisons in prior work suggest that systems with global memory produce more coherent and efficient dialogues than those in which each agent remembers only its own local history [22].

D. Communication Topology and Debate-Style Frameworks

Beyond turn-taking and memory, several works explore how communication topology affects multiagent performance. Li et al. [6] study a multiagent debate (MAD) framework with a *sparser communication topology*, where agents communicate only with select peers in a structured pattern rather than all-to-all. This sparsification reduces communication overhead and can speed up convergence, but also limits the richness of coordination: fewer connections mean fewer opportunities for agents to directly correct or build on each other's contributions. Their results illustrate a trade-off: restricting communication can reduce redundancy but may also limit the system's ability to integrate diverse perspectives and cross-check intermediate results.

More broadly, there is a tension between fully connected topologies (where all agents see the same shared context) and sparser, efficiency-oriented topologies. Fully connected designs maximize the information each agent can access but can suffer from verbosity and coordination overhead. Sparser designs mitigate overhead but risk losing nuanced interactions and cross-checking opportunities. Debate-style frameworks demonstrate that carefully structured communication can yield strong performance on certain reasoning benchmarks [24], yet they may not generalize as well to multi-step, high-coupling tasks that require tight coordination across heterogeneous roles.

E. Summary of Gaps and Positioning of COSMIC

Taken together, the literature highlights several recurring pain points in multiagent LLM collaboration.

1) *Coordination structure:* Sequential and hierarchical frameworks improve reliability but are often too rigid for rich, multidirectional collaboration, while fully decentralized group chats are flexible but fragile in the face of faulty or misaligned agents [5], [7], [8], [9], [10]. Many systems lack an *adaptive leader* that continuously monitors progress, updates the work division, and decides when to stop.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

2) *Turn-taking and termination:* Existing group-chat systems frequently rely on simple round-robin or agent-driven speaker selection and heuristic stopping rules, which can lead to self-invocation loops, imbalanced participation, endless conversations, or premature termination [1], [2], [6], [7], [12].
3) *Shared state:* Many frameworks provide only limited shared context or distributed memory, making it difficult to maintain a consistent global state over long conversations, especially for tasks with strict constraints or long-horizon dependencies [13], [14], [15]. Without a robust shared memory, systems risk fragmentation, repetition, and contradictions across agents.
4) *Communication topology:* Sparse topologies reduce overhead but limit direct cross-checking and rich coordination, whereas fully connected topologies require additional mechanisms to manage verbosity and keep discussions focused [24].

COSMIC is designed explicitly around these gaps. It combines 1) a *leader agent* that provides global oversight, periodically summarizes progress, and decides termination based on task completion rather than a fixed turn budget; 2) a *SSA* that serves as an external arbiter of turns, preventing self-invocation loops and enforcing balanced participation; and 3) a rolling *shared memory buffer* that exposes the last N messages to all agents, ensuring a consistent local view of the evolving conversation. This integrated design aims to retain the flexibility of group-chat–style systems such as AutoGen while addressing their failure modes—unstable turn-taking, weak termination, and fragmented context through centralized oversight and structured memory sharing. Unlike fixed manager-worker workflows, COSMIC externalizes both turn-taking and termination into explicit, framework-level policies (SSA scheduling and a leader termination predicate), enabling reproducible coordination behavior that is not delegated to ad hoc per-agent heuristics.

Finally, COSMIC is intended for high-coupling, high-cohesion tasks where agents must repeatedly condition on each other's outputs rather than simply executing independent subtasks. Our experiments show that a team size of around five agents offers a favorable trade-off between diversity and coordination overhead: larger teams incur diminishing returns and communication costs, whereas smaller teams may lack sufficient specialization. In the remainder of this article, we empirically demonstrate that COSMIC's leader–SSA–memory combination yields more stable, high-quality collaboration on complex tasks than existing sequential, hierarchical, group-chat, and debate-style baselines.

III. Methodology

A. Architectural Overview

Our framework extends the AutoGen *GroupChat* architecture with three components.

1) *Leader agent* (L), which orchestrates multi-agent collaboration and decides termination.
2) *Shared memory buffer* (M), a rolling log of the most recent N messages visible to all agents.

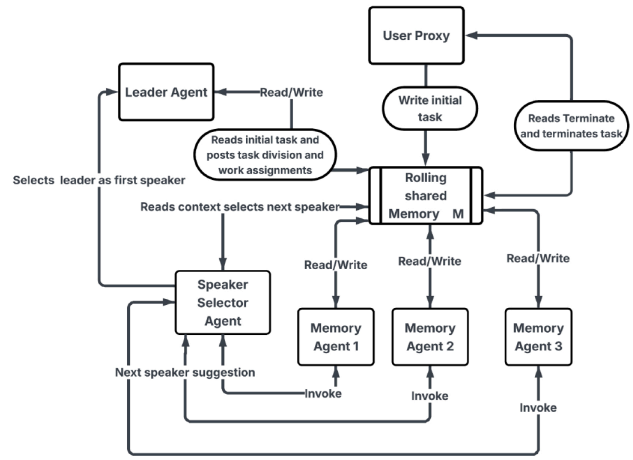

Fig. 1. System architecture of the proposed leader-driven framework.

3) *SSA*, which selects the next speaker after each message is appended to M.

As shown in Fig. 1, the user proxy receives the user query and writes the initial task description into M. At $t = 0$, SSA selects the Leader as the first speaker; the Leader reads from M, emits a structured task brief, and appends it to M. All agents, including the leader, subsequently read from and write to M. We index turns by agent utterances: $t = 0$ denotes the first agent message (the Leader task brief), and the user proxy initialization is not counted as a turn.

User Proxy: For completeness, we include in Fig. 2 a *User Proxy* component. This is a thin wrapper that interfaces with the human user and the underlying framework: it receives the user query, writes the initial task message into the shared memory M, and, at the end of the interaction, reads the Leader's final answer and termination marker from M and returns the result to the user as shown in steps 1 and 11. The user proxy is not itself an LLM agent and is not evaluated; it simply mediates I/O between the user and COSMIC.

B. Leader Agent: Task Decomposition and Termination

The *Leader Agent* L has two primary responsibilities: 1) decomposing the user task into subtasks and assigning them to specialists; and 2) deciding when the conversation has converged sufficiently to produce a final answer and terminate. Unlike a fixed, hard-coded planner, L is implemented as an LLM-driven agent that operates over the shared memory M_t and a known set of specialist agents and roles as shown in steps 3, 4, and 10 of Fig. 2.

1) Task Decomposition and Role Assignment: We assume the Leader knows the set of specialist agents and their roles in advance

$$\mathcal{A}_{\text{spec}} = \{A_1, \ldots, A_m\}, \quad \text{role}(A_i) \in \mathcal{R}. \tag{1}$$

Given the user query q and $\mathcal{A}_{\text{spec}}$, the Leader implicitly defines a task decomposition

$$\mathcal{D} = \{(g_i, a_i, \text{deps}_i)\}_{i=1}^{K} \tag{2}$$

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

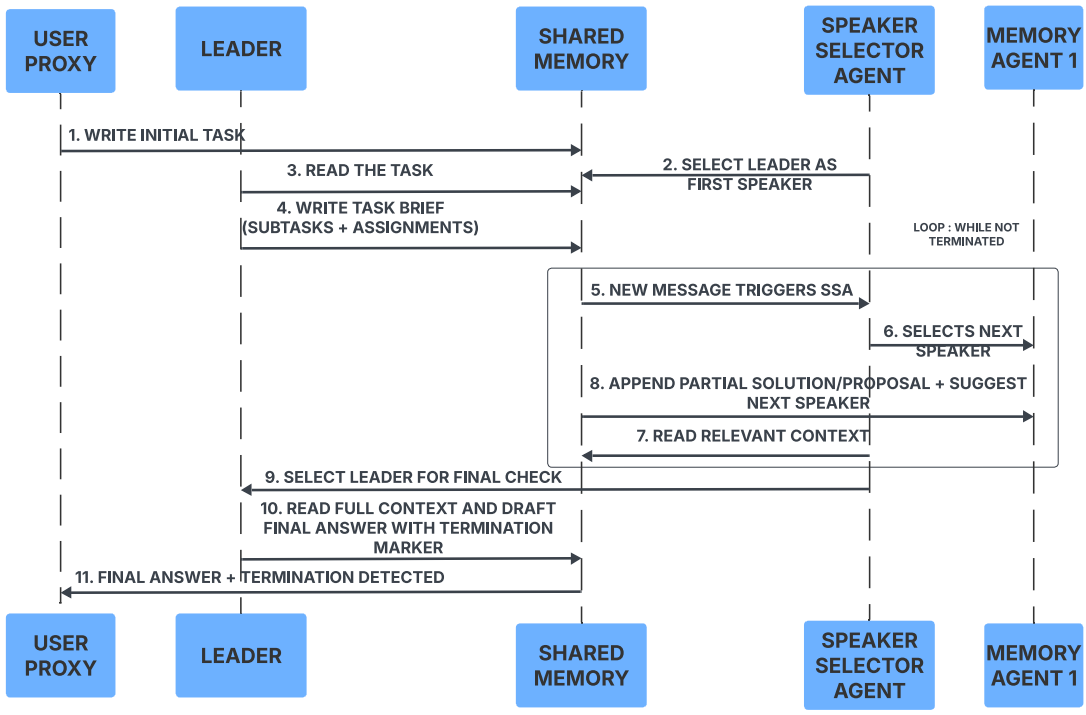

Fig. 2. Sequence diagram illustrating the interaction flow within the COSMIC framework. The process begins with the user proxy initializing the task in shared memory. The leader creates a task brief, after which the SSA manages the conversation loop by selecting the next speaker based on context and fairness. The cycle continues until the leader detects task completion and issues a termination marker.

where g_i is a natural-language description of subtask i (e.g., "derive the algebraic solution", "check unit consistency", "enforce word limit"); $a_i \in \mathcal{A}_{\text{spec}}$ is the specialist assigned to g_i; and $\text{deps}_i \subseteq \{1, \ldots, K\}$ encodes prerequisite subtasks.

Operationally, when SSA selects L at $t = 0$ (Algorithm 1, lines 1–2), the Leader receives q together with a structured description of the specialists (names and roles). It is then prompted to emit a *task brief* that instantiates $\mathcal{D}$ in a machine-readable schema (e.g., a JSON-like list of subtasks, assignees, and dependencies), accompanied by a short natural-language explanation. This brief is written into the shared memory M_0 and serves as a global reference for what constitutes completion of the task.

Subsequent agent turns are encouraged (via their prompts) to explicitly reference the relevant subtask identifiers (e.g., "I am addressing subtask g_2: verify the derivation"), which allows both SSA and the Leader to track which subtasks are pending, in progress, or complete.

2) Termination Predicate: During the conversation, the Leader is periodically invoked by SSA through the heartbeat mechanism: whenever $t > 0$ and $t \bmod T_{\text{leader}} = 0$ and the most recent speaker is not the Leader ($S_t \neq L$), Algorithm 1 schedules a Leader heartbeat turn. On each such heartbeat turn, the Leader reads the shared memory M_t and evaluates whether the conversation can safely terminate.

For each subtask g_i, the Leader maintains a latent status

$$s_i(t) \in \{\text{pending}, \text{in-progress}, \text{complete}\}. \tag{3}$$

From these, it computes a *coverage score*

$$c_i(t) = \mathbb{I}[s_i(t) = \text{complete}], \qquad C(t) = \frac{1}{K}\sum_{i=1}^{K} c_i(t) \tag{4}$$

so that $C(t) \in [0, 1]$ is the fraction of required items that appear complete given the current memory M_t.

In tasks without an explicit multiitem checklist, K is treated as the number of required constraint classes and schedule components specified in the task brief (e.g., each agent's constraints and global capacity), and $C(t) = 1$ denotes that all such components have been explicitly addressed in M_t.

In addition, the Leader performs a structured consistency assessment over M_t using a checklist prompt that asks whether the partial solution violates any specified constraints (e.g., contradictory diagnoses, incorrect genre or word limits, inconsistent numerical steps)

$$H(t) \in [0, 1] \tag{5}$$

where $H(t) = 1$ indicates that no major inconsistencies are detected and $H(t) = 0$ indicates severe problems. In practice, $H(t)$ is obtained by asking the Leader to rate overall consistency on a fixed scale (e.g., 0–10) and rescaling to $[0, 1]$.

To avoid premature termination, the Leader also enforces a *stability* condition on these scores between two consecutive heartbeat turns

$$\left|C(t) - C(t - T_{\text{leader}})\right| \leq \varepsilon_C, \qquad \left|H(t) - H(t - T_{\text{leader}})\right| \leq \varepsilon_H \tag{6}$$

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Algorithm 1: SSA_Select_Next_Speaker.

Input: shared memory M_t, recency vector $r_t(\cdot)$, Leader L, agents $\{A_1, \ldots, A_m\}$, heartbeat period T_{leader}
Output: next speaker $S_{t+1} \in \mathcal{A}$

```
if t = 0 then
    return L                    // Leader always starts
end
// Force Leader heartbeat (unless Leader just spoke)
if t > 0 and t mod T_leader = 0 and S_t ≠ L then
    return L
end
last_speaker ← S_t
nomination ← parse_nomination(last_speaker, M_t)
candidates ← A \ {last_speaker}          // no immediate self-repeat
// Rule 1: respect a valid nomination
if nomination ∈ candidates then
    return nomination
end
// Rule 2: context-based priority
active_subtask ← infer_current_subtask(M_t)
relevant ← {a ∈ candidates : role(a) matches active_subtask}
if relevant ≠ ∅ then
    return arg max_{a∈relevant} r_t(a)   // pick most idle relevant agent
end
// Rule 3: fairness fallback
return arg max_{a∈candidates} r_t(a)     // pick most idle agent overall
```

for small thresholds $\varepsilon_C, \varepsilon_H > 0$ (we use 0.1 unless otherwise stated).

We then define a binary termination predicate

$$\tau(M_t, \mathcal{D}) = \begin{cases} 1, & \text{if } C(t)=1,\ H(t)\geq\theta_H, \text{ and stability holds} \\ 0, & \text{otherwise} \end{cases} \tag{7}$$

for a chosen consistency threshold $\theta_H \in (0, 1)$ (we use $\theta_H = 0.8$ unless otherwise stated). Intuitively, $\tau(M_t, \mathcal{D}) = 1$ only when.

1) All planned subtasks are marked *complete* by the Leader under the task brief ($C(t) = 1$).
2) The current solution passes a structured global consistency checklist ($H(t) \geq \theta_H$).
3) Both coverage and consistency are stable across two consecutive Leader heartbeats.

Termination hyperparameters: selection and sensitivity: Unless otherwise stated, COSMIC uses $\theta_H = 0.8$, $\varepsilon_C = \varepsilon_H = 0.1$, and $T_{\text{leader}} = 7$. These defaults were selected via a small pilot sweep on held-out prompts to balance premature termination (ending before contradictions are resolved) against coordination overhead (additional leader checkpoints and turns). θ_H controls the minimum global consistency required to terminate: lower values increase early exits while unresolved conflicts remain; higher values increase interaction cost with diminishing returns. ε_C and ε_H prevent termination under oscillatory behavior across consecutive heartbeats: larger thresholds admit unstable convergence, while smaller thresholds can delay termination unnecessarily. T_{leader} sets the Leader checkpoint frequency: smaller values increase oversight but add overhead; larger values reduce overhead but increases drift risk between checkpoints. A one-factor-at-a-time sensitivity sweep around these defaults shows smooth performance variation, with the chosen configuration near a favorable quality–overhead trade-off region: more frequent checkpoints increase message overhead, while less frequent checkpoints or looser stability thresholds elevate failure risk (Appendix A).

On each heartbeat turn, the Leader operates as follows.

1) Update the estimated statuses $s_i(t)$ and compute $C(t)$ and $H(t)$.
2) If $\tau(M_t, \mathcal{D}) = 1$, compose a final user-facing answer by aggregating the relevant messages in M_t, write this answer and a termination marker into M_t, and signal the end of the session.
3) Otherwise, post a *progress update* summarising which subtasks are incomplete or inconsistent and explicitly directing the appropriate specialists (e.g., "Editor should now enforce the 500-word limit", "Verifier should recheck step 3 of the derivation").
4) Once the Leader has written the final answer and termination marker into M_t, the User Proxy (Fig. 1) detects this marker, retrieves the answer from M_t, and returns it to the human user, thereby closing the session.

Empirically, this combination of task decomposition, periodic heartbeat turns via SSA, and a principled termination predicate prevents the system from drifting indefinitely or terminating too early, while still allowing rich, flexible collaboration among specialist agents.

C. Shared Memory Buffer

At the core of COSMIC is the shared memory buffer M, which facilitates inter-agent context sharing. M is implemented as a first-in, first-out (FIFO) rolling log that stores the last N messages exchanged among agents.

1) Cache Replacement and Token Management: To ensure the prompt length remains within the LLM's context window while preserving the most immediate conversational history, we employ a strict rolling window policy. After each agent produces an output, the message is appended to M. If the buffer size exceeds N messages, the oldest entry (index 0) is dropped.

This FIFO mechanism serves as our primary token management strategy. By fixing N (typically tuned between 8 and 12 depending on task complexity), we ensure that the prompting context grows on the order of $\mathcal{O}(N \times L)$, where L is the average message length. This prevents the context overflow issues

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

common in infinite-history chat logs while avoiding the complexity and potential context-fragmentation of least recently used (LRU) or summary-based compression methods.

Fig. 1 illustrates this setup: the Leader Agent initiates the task by posting an initial task breakdown and work assignment in rolling shared memory, and all content agents read from and write to this rolling buffer, ensuring every agent has immediate access to the most recent N turns without needing to query a separate database.

D. Memory Agent

To interface cleanly with the shared buffer M, we implement each specialist as a *memory agent* by extending AutoGen's `ConversableAgent`. Concretely, as shown in Fig. 1, before generating each response, the agent automatically: 1) reads the current rolling context M_t; 2) prepends the Leader's task brief (when available); and 3) conditions its prompt on both the task brief and the most recent N messages. This design ensures that all agents operate over a consistent, up-to-date view of the collaboration state without requiring external retrieval infrastructure. We treat the Leader's task brief as a pinned artifact stored outside the FIFO window and prepended to each agent prompt.

E. SSA

In COSMIC, the SSA is the sole component responsible for deciding which agent speaks next at each turn. All turn-taking behavior, including the periodic "leader heartbeat", is implemented inside SSA. Neither the Leader nor any specialist agent can directly schedule itself; they may only *nominate* the next speaker in their messages, which SSA may or may not follow.

Let $\mathcal{A} = \{L, A_1, \ldots, A_m\}$ denote the set of agents, where L is the Leader and A_i are specialist content agents. At turn t, the shared rolling memory $M_t = (m_1, \ldots, m_t)$ contains the last N messages. SSA maintains.

1) $S_t \in \mathcal{A}$: the speaker at turn t.
2) A recency vector $r_t(a)$ giving the number of turns since agent a last spoke.
3) A role label $\mathrm{role}(a)$ for each agent (e.g., `MathSolver`, `Verifier`, `WorldBuilder`).
4) A global turn counter t. We define t strictly as the index of generated agent utterances, excluding the static User Proxy initialization.

We define a *leader heartbeat period* $T_{\text{leader}} = 7$. SSA uses the following selection rule: at the very first turn ($t = 0$), SSA always selects the Leader as the initial speaker so that the conversation begins with a task brief and subtask decomposition. During the conversation, SSA enforces: 1) a heartbeat that forces the Leader to speak every T_{leader} turns; 2) an antihijacking constraint that prevents immediate self-repetition; and 3) a preference for contextually relevant but under-utilised agents. Formally, SSA implements.

F. End-to-End Interaction Flow

Fig. 2 complements the static architecture in Fig. 1 by showing the temporal interaction between the User Proxy, shared memory M, the SSA, the Leader, and the specialist agents.

Initialisation: The User Proxy receives the human query and writes the initial task message into M (step 1). At $t = 0$, the SSA selects the Leader as the first speaker (step 2). The Leader reads the task from M, constructs a structured task brief with subtasks and role assignments, and writes this brief back into M (steps 3–4).

Collaborative loop: After each agent turn, the SSA is invoked when a new message appears in M (step 5). The SSA reads the current context, applies the nomination, antihijacking, and fairness rules described in Section III-E, and selects the next speaker (step 6). The selected specialist agent reads the relevant context from M and appends its partial solution or proposal (steps 7–8). This loop repeats while the termination predicate remains false; every T_{leader} turns, the SSA forces a Leader heartbeat.

Termination: On a scheduled Leader turn, the Leader evaluates $\tau(M_t, \mathcal{D})$. If $\tau(M_t, \mathcal{D}) = 1$, the Leader writes the final answer and a termination marker into M (step 10). The User Proxy then detects this marker and returns the answer to the user (step 11).

IV. Problem Formulation for Framework Comparison

We consider multiagent problem solving scenarios in which a set of agents, each possessing different expertise or computational abilities, collaborate to produce a final solution. Let there be m specialist agents $\{A_1, \ldots, A_m\}$, a Leader agent L, and a User Proxy U. All agents read from and write to a shared memory buffer M that stores the last N messages.

To categorize our tasks we define two terms below.

1) *Coupling:* Reflects the interdependence between agents, which refers to how strongly the agents rely on each other's output. High coupling means strong coordination between agents, whereas low coupling refers to more independent work without much reliance on each other's input.
2) *Cohesion:* Measures the extent of collaboration between agents, which is the degree to which agents refine and integrate their outputs in a coherent manner toward a high-quality solution. High cohesion signifies strong collaboration yielding refined results, whereas low cohesion often produces degraded outputs.

Illustrative interpretation (graph analogy): The notions of cohesion and coupling can be *illustrated* using a graph view of collaboration. Consider a graph whose nodes are agents and whose edges represent interaction links (e.g., information exchange, dependency on an intermediate output, or corrective feedback). In this analogy, *cohesion* corresponds to dense, reinforcing interaction within a working group, while *coupling* corresponds to strong cross-dependencies between different roles or subparts of the task.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE I
COHESIVENESS AND COUPLING COMPARISON ACROSS TASKS

Task	Cohesiveness (ρ)	Coupling (σ)	Reasoning
Bandwidth allocation	0.70	0.75	Shared global constraint + negotiation dependencies
Multilingual translation	0.75	0.50	Integration needed, but per-language spans mostly independent
Short story generation	0.86	0.70	Single artifact + iterative cross-agent revisions
Healthcare diagnosis	0.88	0.80	Mid-execution update forces joint re-evaluation.
Mathematical problems	0.85	0.82	Multi-step refinement with strong interdependence of partial solutions.
Robot Warehouse Coord.	0.90	0.89	Hard shared constraints + continuous coordination

A common proxy for within-group interaction density is

$$C_{\text{intra}} = \frac{2E_{\text{intra}}}{N_{\text{intra}}(N_{\text{intra}} - 1)} \tag{8}$$

where E_{intra} is the number of within-group edges and N_{intra} is the number of agents in the group. A proxy for cross-group dependence is

$$C_{\text{inter}} = \frac{E_{\text{inter}}}{N_{\text{inter}}^2} \tag{9}$$

where E_{inter} counts cross-group edges and N_{inter} is the total number of agents involved. We emphasize that these graph quantities are *illustrative*: in this work, we do *not* compute ρ and σ from logged interaction graphs. Instead, we assign ρ and σ *a priori* per task using the fixed rubric described below.

Significance: Cohesion and coupling capture different stressors for multiagent coordination.

1) *High coupling* means agents must repeatedly condition on each other's intermediate outputs to proceed (e.g., negotiation, joint planning, constraint reconciliation). Coordination failures (lost constraints, inconsistent subplans) are more costly.
2) *High cohesion* means the task requires iterative integration and revision into a single coherent artifact (e.g., schedules, diagnoses, narratives), rather than independent outputs that can be concatenated.

Tasks that are *high-coupling and high-cohesion* therefore place the greatest demands on shared context, structured turn-taking, and reliable convergence, and serve as the strongest stress-tests for coordination frameworks.

Based on these terms, we categorize our tasks in the following ways.

1) Loosely coupled tasks with low cohesiveness (normal tasks with minimal collaboration).
2) Loosely coupled tasks with high cohesion (tasks requiring limited but meaningful integration).
3) Highly coupled tasks with high cohesion (tasks where coordination and collaboration are essential).

All the tasks used for comparing the frameworks are listed below in Table I with their respective degrees of coupling and cohesion.

Role of coupling and cohesion metrics: Coupling and cohesion are used as a task-characterization layer rather than as inputs to the composite performance score. They motivate benchmark selection and help explain variance in framework performance: tasks with higher coupling/cohesion (Table I) demand stronger coordination mechanisms, and these are where COSMIC's gains over baselines are most pronounced.

How ρ and σ are assigned: The cohesiveness ρ and coupling σ values in Table I are task-characterization scores in $[0, 1]$ assigned before running any framework, using a fixed internal rubric. σ (coupling) reflects how strongly agents must condition on each other's intermediate outputs to proceed (low: largely independent; high: interdependent multistep negotiation/planning). ρ (cohesion) reflects how much iterative integration and cross-agent refinement is required to produce a coherent final output (low: outputs can be concatenated; high: repeated revision is necessary). These scores are not used in the composite performance metric; they only motivate benchmark selection and help interpret performance variance across task types.

V. ANALYSIS OF MEMORY MODEL

A. Utilization Efficiency

The shared memory design effectively functions as a blackboard system—a common space where agents post intermediate results and track the state of the overall solution. By logging intermediate decisions and constraints in M, we reduce redundant queries and make contradictions easier to detect and resolve via cross-checking in later turns. Whenever an agent needs a piece of information (e.g., a sub-solution or constraint mentioned earlier), it consults M instead of asking another agent anew. This reuse reduces unnecessary back-and-forth compared with purely pairwise message passing.

B. Context Depth Versus Quality

We observe that deeper memory (larger N) initially increases solution quality: agents make fewer omissions because they can reference more prior context. However, beyond a certain depth, the returns diminish. If N exceeds the typical length needed to cover all distinct points of the discussion, additional memory mainly introduces verbosity or the risk of overfitting to early turns (e.g., adhering too rigidly to an initial idea).

In all experiments, we use a rolling shared-memory buffer of the last N messages visible to every agent. A larger N preserves more global context but increases the risk of token overflow and redundant reasoning, while a smaller N improves efficiency at the cost of potentially forgetting earlier commitments. We therefore select N per task based on its coordination horizon and context needs.

1) *Mathematics:* $N = 10$ (multi-step derivations and cross-checking between specialists).
2) *Multilingual translation:* $N = 8$ (moderate shared context required for consistency).
3) *Story writing:* $N = 12$ (long narrative continuity and cross-agent revisions).
4) *Healthcare diagnosis:* $N = 10$ (diagnostic reasoning chains across specialists).

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

5) *Resource scheduling:* $N = 8$ (negotiation-focused, short- to medium-horizon).
6) *Robot warehouse coordination:* $N = 12$ (multistep planning and constraint tracking).

We chose these N values via a small pilot sweep per task (varying N over 6, 8, 10, 12) to balance constraint retention against message/token overhead.

These values were chosen to balance context preservation against computational efficiency; all other framework settings (Section VIII) are held fixed while varying only the coordination topology.

Our observations align with the memory-sharing approach proposed by Gao and Zhang [13], which demonstrated that sharing a dynamically growing memory pool improves performance on open-ended tasks. However, unlike their static or ever-growing logs that require complex retrieval mechanisms, our rolling-window approach minimizes latency and token costs by enforcing a strict relevance horizon N, and in our setting does not require a separate retrieval stage, since relevance is enforced by the rolling horizon.

C. Semantic Consistency Under Multiagent Collaboration

In COSMIC, *semantic consistency* means that all agent contributions remain aligned with the user's constraints and with each other over time: the final answer should not contradict earlier assumptions, change the problem interpretation midway, or drift away from specified requirements (e.g., genre, tone, or numerical constraints). COSMIC promotes semantic consistency through three interacting mechanisms.

1) Task brief and constraint anchoring: At $t = 0$, the Leader constructs a structured task brief $\mathcal{D}$ that explicitly encodes the user's constraints (e.g., "500-word cyberpunk story in first person", "do not change the diagnosis", "translate into formal Spanish") together with a decomposition into subtasks and roles. This brief is written into the shared memory and is prepended or referenced in the prompts of all specialist agents. As a result, every agent sees the same global specification and is encouraged to check its local actions against this shared set of constraints.

2) Shared rolling memory and cross-checking: All agents, including the Leader and SSA, read from a rolling shared memory buffer M_t that stores the recent interaction history. When a specialist acts, its prompt explicitly conditions on.

a) the user query and task brief,
b) relevant prior agent outputs (e.g., solver steps, earlier story segments), and
c) any known constraints or decisions recorded so far.

This gives agents a unified view of the evolving solution and allows them to perform local consistency checks (e.g., a verifier can recheck earlier equations; an Editor can enforce the requested tone). Because the memory is shared, inconsistencies introduced by one agent can be detected and corrected by others in later turns.

3) Leader heartbeat and global consistency checks: On every heartbeat turn (when SSA selects the Leader), the Leader recomputes the coverage $C(t)$ and consistency $H(t)$ scores from Section III-B. The consistency score $H(t) \in [0, 1]$ is obtained by prompting the Leader with a structured checklist, asking it to detect.

a) contradictions between different agent proposals,
b) violations of user-specified constraints (e.g., wrong genre, tense, or length), and
c) missing justifications or gaps in reasoning.

If $H(t)$ falls below a threshold or changes sharply compared with the previous heartbeat, the Leader does *not* terminate. Instead, it posts a progress update that explicitly flags the inconsistency and assigns corrective subtasks to appropriate specialists (e.g., asking the Verifier to re-check step 3 of a derivation, or asking the Editor to enforce the word limit). The termination predicate $\tau(M_t, \mathcal{D})$ only returns 1 when coverage is complete and $H(t)$ remains above a threshold across two consecutive heartbeats, reducing the likelihood that the final answer violates major constraints.

Illustrative example (story generation): Consider the story-generation task where the user requests an "approximately 2000-word adventure-thriller story." The Leader's task brief encodes.

a) *Constraints:* genre = adventure-thriller, length $\approx$ 2000 words, *maintain narrative coherence and character consistency*;
b) *Subtasks:* Character development, world-building, plot outline, drafting, and editing for coherence and length.

During the first few turns, specialist agents propose characters, setting, and plot structure. Suppose a Drafting agent introduces a subplot that contradicts an earlier established constraint (e.g., changes the central motivation of the protagonist or breaks timeline continuity). On the next heartbeat, the Leader scans M_t, detects the contradiction, and lowers $H(t)$. Instead of terminating, it posts an instruction such as.

> *"The current draft conflicts with the agreed plot timeline and character motivation. Plot and Editor agents should revise the affected sections to restore consistency while preserving the established outline and target length."*

Agents then revise accordingly. Only when the Leader observes that the story respects the task constraints and that $H(t)$ has stabilized does $\tau(M_t, \mathcal{D})$ evaluate to 1 and the conversation terminate.

VI. Evaluation Methodology

We evaluate COSMIC on a fixed suite of coordination tasks designed to stress turn-taking, shared context, and convergence under a bounded interaction budget. To quantify robustness under the stochasticity of LLM decoding, we run COSMIC for $K = 5$ independent trials using fixed decoding hyperparameters (temperature $\tau = 0.7$). For each baseline framework, we run $N = 5$ independent trials under the same task prompts, interaction budget, model configuration, and decoding settings wherever the framework permits comparable controls.

For COSMIC, we report mean $\pm$ standard deviation over $K = 5$ trials. For baselines, we report 1) the validity rate under task-specific hard-constraint criteria (including termination

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

within the fixed interaction budget); and 2) the *best validated* composite score among valid trials as an optimistic (baseline-favoring) upper bound. This presentation separates *reliability* (validity rate) from *peak performance* (best validated score) in high-coupling settings where non-convergence is common.

If a framework produces zero valid trials for a task, we report the score as 0 (no valid run) and the runtime as time-to-budget exhaustion.

1) *Validity criteria:* A trial is considered *valid* if it: 1) terminates within the fixed interaction budget; and 2) satisfies the task's hard constraints (defined in the task specification and rubric for that task). Hard constraints include task-dependent requirements such as priority ordering, completeness of requested outputs, and adherence to explicit formatting or feasibility constraints.
2) *Composite performance metric:* For each trial, we compute a composite performance score $S \in [0, 100]$ as a weighted sum of normalized subscores. While the concrete constraints differ by task domain, the scoring structure and weights are held fixed across tasks for consistency.

Correctness ($w_1 = 0.50$): Let c be the number of satisfied hard constraints for the task, and let $C_{\max}$ be the total number of hard constraints defined for that task

$$S_{\text{corr}} = 100 \times \frac{c}{C_{\max}}. \tag{10}$$

Solution Completeness ($w_2 = 0.20$): Let u be the number of distinct specialist agents that contribute nontrivial task-relevant content, and let $U_{\max}$ be the total number of specialist agents defined for that task (excluding the User Proxy and any non-content instrumentation roles)

$$S_{\text{comp}} = 100 \times \frac{u}{U_{\max}}. \tag{11}$$

Reasoning Quality ($w_3 = 0.15$): Let $r \in [1, 10]$ be a rubric-based reasoning quality rating applied consistently across all trials of the same task (e.g., coherence of derivation steps, absence of logical gaps)

$$S_{\text{reason}} = 10 \times r. \tag{12}$$

Convergence Efficiency ($w_4 = 0.10$): Let T denote the number of generated agent turns in the trial (one turn equals one generated agent message; the static User Proxy initialization is excluded). We penalize excessive verbosity using an exponential decay

$$S_{\text{conv}} = \max\left(0,\ 100 \times e^{-0.05 \cdot T}\right). \tag{13}$$

Collaboration Quality ($w_5 = 0.05$): Let $r_{\text{collab}} \in [1], [10]$ be a rubric-based rating of collaboration quality (e.g., effective hand-offs, conflict resolution, integration of specialist outputs)

$$S_{\text{collab}} = 10 \times r_{\text{collab}}. \tag{14}$$

Final Score

$$\text{Score} = w_1 S_{\text{corr}} + w_2 S_{\text{comp}} + w_3 S_{\text{reason}} + w_4 S_{\text{conv}} + w_5 S_{\text{collab}}. \tag{15}$$

Here, $C_{\max}$ denotes the number of hard constraints defined for the task rubric, and $U_{\max}$ denotes the number of specialist agents defined for that task (excluding the User Proxy and any instrumentation-only roles).

3) *Execution Time and Time Score*: We report raw wall-clock time (seconds) and a normalized *Time Score*. Let $T_{\text{framework}}$ be the mean wall-clock time for a framework on a given task (including time-to-budget exhaustion for invalid runs), and let $T_{\min}$ be the fastest mean time recorded among all evaluated frameworks for that task

$$\text{TimeScore} = 100 \times \frac{T_{\min}}{T_{\text{framework}}}. \tag{16}$$

4) *Baseline Frameworks*: We compare COSMIC against widely used multiagent or multirole LLM frameworks spanning dominant coordination paradigms.

a) *CrewAI (Sequential / Hierarchical):* Manager–worker pipelines with sequential and hierarchical flows [8]. Versions: crewai v0.35.8 and crewai-tools v0.4.6.
b) *AutoGen GroupChat:* Leaderless group-chat coordination in a shared channel [7]. Versions: autogen v0.3.1 and autogen-agentchat v0.2.37. Since the default leaderless configuration frequently fails to converge on high-coupling tasks under a fixed interaction budget, we also report an auxiliary strengthened variant (*AutoGen + Minimal Coordinator*) implemented by adding a lightweight coordination role to encourage turn discipline and trigger reconciliation when contradictions persist. Both AutoGen variants are evaluated under identical prompts, budgets, and decoding settings.
c) *LangChain Pipelines:* structured chain- and tool-based multistep reasoning used as a minimal orchestration baseline [10]. Versions: langchain v0.1.20, langchain-core v0.1.52, and langchain-community v0.0.38.
d) *OpenAI Swarm:* A stateless orchestration framework emphasizing lightweight routing and autonomous agents [25].
e) *Multiagent Debate (MAD):* A debate-style architecture with sparse and fully connected topologies, evaluated on GSM8K [6].

These baselines are representative of manager–worker hierarchies, group chat, pipeline orchestration, stateless routing, and debate-based coordination, providing a broad comparison spectrum against COSMIC's leader-driven shared-memory design. A qualitative comparison of COSMIC with representative frameworks is summarized in Table II.

VII. Finding the Optimum Number of Agents

In principle, one can scale the number of agents in a collaborative problem-solving setting indefinitely. Many tasks—including complex business decision-making, strategic planning, and general-purpose language tasks—could, in theory, benefit from a greater breadth of expertise and perspectives. However, increasing the team size introduces practical trade-offs. As the number of agents grows, the volume of interagent communication rises, increasing computation time and delaying

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE II
QUALITATIVE COMPARISON OF COSMIC WITH REPRESENTATIVE FRAMEWORKS (ARCHITECTURE + OBSERVED BEHAVIORS IN OUR EVALUATION; SEE APPENDIX B)

Feature	COSMIC	CrewAI Sequential	CrewAI Hierarchical	AutoGen	LangChain	OpenAI Swarm
Multiagent framework	✓ (Specialist agents)	✓ (Linear pipeline)	✓ (Manager–worker)	✓ (Group chat)	✓ (Chains / routing)	✓ (Agent routing)
Agent freedom	✓ (Autonomy + oversight)	✘ (Fixed order)	✘ (Manager-mediated)	✓ (Free interaction)	Partial (Structured flow)	✓ (High autonomy)
Leader-driven coordination	✓ (Explicit Leader)	✘ (None)	✓ (Manager role)	Partial (Manager component)	✘ (None)	✘ (None)
Speaker selection mechanism	✓ (SSA arbiter)	✘ (Fixed order)	✘ (Manager calls only)	Partial (GroupChatManager)	✘ (Static order)	✘ (No turn arbiter)
Shared memory context	✓ (Rolling buffer M)	Partial (Context pass-through)	Partial (Context via manager)	Partial (Shared transcript)	✘ (External state)	✘ (Stateless by default)
Task convergence reliability	✓ (Consistent closure)	✘ (Low validity rate)	✘ (Low validity rate)	Partial (Mixed validity)	✘ (Mixed / fragile)	Partial (Mixed validity)
Performance on reasoning tasks	✓ (High across tasks)	✘ (Often incomplete)	✓ (Task-dependent)	✘ (Imbalance observed)	✘ (Output variability)	Partial (Task-dependent)
Adaptability across domains	✓ (Role modularity)	✓ (Chain modularity)	✓ (Worker modularity)	✓ (Role prompts)	✓ (Composable chains)	✓ (Composable agents)
Scalability	✓ (Add specialists)	✓ (Add steps)	Partial (Manager bottleneck)	Partial (Chat overhead)	Partial (Flow-dependent)	✓ (Router-based)

convergence. Beyond a certain point, the added complexity yields diminishing returns or can even degrade performance.

To identify an effective team size for our leader-guided, context-rich coordination framework, we conducted empirical studies on representative tasks while varying the number of specialist agents from 2 to 10. Across these experiments, we observed a consistent pattern: performance peaks around five agents and then gradually declines as more agents are added (Fig. 3). This improvement comes with increased coordination overhead; total messages and completion time grow superlinearly beyond the empirical optimum (Fig. 4). This behavior follows from how COSMIC's architectural components (SSA, Leader, and shared memory) trade off role coverage against coordination overhead.

Role coverage versus redundancy: The tasks we consider naturally decompose into a small number of conceptual roles (e.g., solver, verifier, planner, stylist/editor, and domain expert). With very small teams (2–3 agents), some roles must be overloaded: the same agent is forced to both derive a solution and verify it, or to reason and then rewrite the final output. This increases the likelihood of uncorrected mistakes and lowers overall quality.

Around five agents, we typically have one specialist per major role. At this point, the Leader can assign subtasks to cover the full reasoning pipeline without excessive overlap, and the SSA can schedule turns so each specialist has sufficient opportunity to contribute. Adding further agents beyond this regime rarely introduces genuinely new capabilities; instead, it primarily creates *redundant* roles that duplicate existing expertise. In our logs, these extra agents often repeat analyses already performed by others or contribute brief, low-impact comments.

Single-threaded coordination overhead: COSMIC's turn-taking is intentionally single-threaded: at any step, the SSA

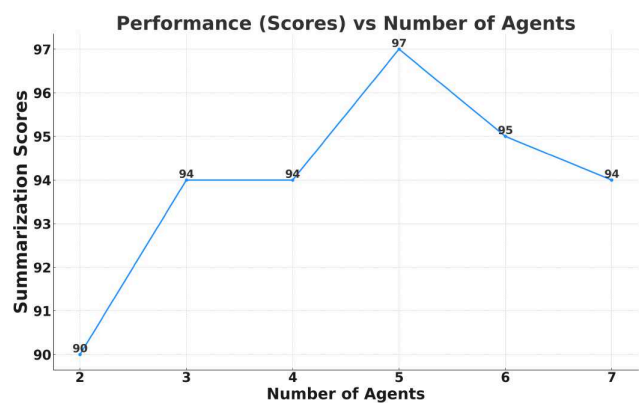

Fig. 3. Performance scores in the scaling experiment (summarization) across different agent configurations. Performance peaks at five agents, illustrating the balance between diversity and overhead.

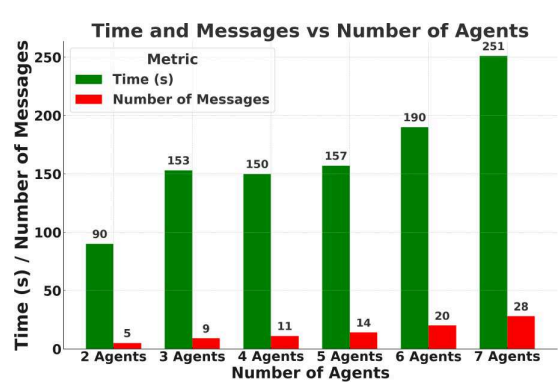

Fig. 4. Total messages and completion time in the scaling experiment (summarization). Time grows superlinearly as agent count increases beyond the empirical optimum.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

selects exactly one next speaker while all other agents are idle. Increasing team size therefore does not introduce parallelism; it increases the size of the candidate pool that SSA must schedule. As the number of agents grows, two architectural bottlenecks emerge.

1) *SSA latency:* The number of turns between successive opportunities for a critical specialist to speak increases because SSA must balance fairness and role coverage across a larger pool.
2) *Leader heartbeat cost:* Leader checkpoint turns become more expensive because they must integrate a larger volume of partially redundant contributions before updating task status and evaluating termination.

For teams larger than about five agents, this coordination overhead offsets the modest gains from extra specialization, and quality plateaus or declines.

Context-window pressure and memory truncation: COSMIC uses a fixed-size rolling shared memory window of N messages to maintain global state. As the number of agents grows, the number of messages produced per unit of problem-solving progress increases. Earlier parts of the dialogue—often including core constraints, user requirements, or early derivation steps—are pushed out of the rolling window M_t sooner. This effect is particularly harmful on long-horizon reasoning tasks (e.g., GSM8K-style multi-step math), where later steps must reference earlier assumptions.

Empirically, runs with large teams (e.g., 8–10 agents) are more likely to forget initial constraints because the shared buffer becomes saturated with coordination chatter. In contrast, teams of around five agents strike a balance: large enough to cover major roles, but small enough to retain key context within the rolling memory horizon.

A. Generalized Performance Modeling

While our experiments suggest that approximately five agents yield strong performance for the tasks studied, this number is not universally optimal. To generalize the observed trade-offs, we propose a modular performance function.

We define a predicted performance score P as a product of three components

$$P = P_{\text{max}} \cdot G(m) \cdot H(R) \cdot D(t_{\text{wall}})$$

where P_{max} is the maximum achievable performance, m is the number of specialist agents, R is the message count, and t_{wall} is wall-clock time.

1) *Agent utility ($G(m)$)*

$$G(m) = e^{-\alpha(m - m_{\text{opt}})^2}.$$

This Gaussian term models the trade-off between role coverage (improving performance as m increases) and coordination bloat (decreasing performance as m becomes too large).

2) *Message effectiveness ($H(R)$)*

$$H(R) = \frac{R}{R + k}.$$

This model's diminishing returns: early messages contribute substantially, while additional messages from redundant agents yield diminishing marginal utility.

3) *Time efficiency ($D(t_{\text{wall}})$)*

$$D(t_{\text{wall}}) = \frac{1}{1 + \gamma t_{\text{wall}}}.$$

This penalizes orchestration overhead as wall-clock time increases due to SSA scheduling and Leader integration costs.

This model provides a theoretical lens for the empirical observation that $m_{\text{opt}} \approx 5$ often maximizes performance before coordination overhead and memory pressure begin to dominate.

VIII. Benchmarking and Comparing Different Frameworks

All frameworks are evaluated under identical hardware and decoding settings to ensure a controlled comparison. Unless otherwise stated, experiments run on a single NVIDIA H100 SXM GPU. In our main experiments, we use.

1) Qwen2.5-72B (32k-token context window); and
2) Nemotron (8k-token context window) for selected robustness checks.

Across all tasks and frameworks, we use identical decoding settings (temperature $\tau = 0.7$, default nucleus sampling parameters) and no explicit per-message `max_tokens` cap beyond the model's native context window. All agents within a given framework share the same LLM configuration; only the coordination structure (e.g., leader versus leaderless, shared memory versus stateless) differs.

We benchmark across tasks spanning low-to-high coordination requirements. On low-coupling tasks, COSMIC is competitive with simpler pipelines. As task coupling and required cross-agent refinement increase, COSMIC yields larger gains in both solution quality and reliability, highlighting the importance of explicit turn-taking, shared context, and leader-guided convergence in coordination-heavy settings.

The tasks used for benchmarking have been described below.

A. Multilingual Translation Task

In this challenge, each agent specialized in translating from a single distinct language into English. The source text contains interspersed segments from multiple languages within a single message. Unlike sequential or strictly hierarchical frameworks, our system enabled these language experts to discuss and negotiate translations collaboratively. By leveraging shared context and iterative feedback, the agents were able to produce a more accurate and coherent translation. Other frameworks, constrained by their unidirectional or strictly pipelined message flow, struggled to maintain consistency and clarity, resulting in lower-quality outputs.

B. Story Writing Task

We tasked the agents with composing a 2000-word adventure-thriller story. Each agent served a distinct creative

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

function—such as plot design, character development, world building, and theme definition. While the other frameworks could only elaborate on the initial agent's concept with limited opportunities for revision, our approach allowed agents to continually refine and improve the narrative. By discussing and revisiting earlier decisions until a consensus was reached, COSMIC produced stories with higher overall quality scores.

C. Mathematics Problem Task

In a complex mathematics problem requiring coordinated solutions, each agent specialized in a different domain (e.g., calculus or algebra). The agents were tasked with collectively solving a set of heterogeneous subproblems (algebra, number theory, combinatorics, calculus, and geometry) under a shared coordination budget. COSMIC, with its free-flowing collaboration and shared memory, successfully integrated each agent's partial solution into a coherent whole. Remarkably, the other frameworks failed to converge on a final answer. This result not only emphasizes the necessity of genuine, bidirectional communication for problem-solving but also demonstrates the versatility of COSMIC, which can handle tasks that, at first glance, seem well-suited to a purely sequential approach.

D. Healthcare Task

In this dynamic healthcare scenario, a group of agents, each specializing in a different medical domain received a structured symptom description and initial test signals, followed by a midexecution release of additional lab and imaging results. The agents were tasked with collaboratively forming a detailed diagnosis. Midway through their discussion, new test results arrived, requiring the agents to incorporate the updated information into their ongoing reasoning process.

It was instructive to observe how various frameworks adapted to this evolving situation. Our leader-driven, context-rich framework gracefully integrated the new data into its existing diagnostic reasoning, enabling a seamless refinement of the proposed diagnosis. In contrast, other frameworks based on strictly sequential or hierarchical communication patterns struggled to effectively incorporate the additional test results, often failing to produce a coherent, final diagnostic outcome.

E. Bandwidth Allocation and Resource Scheduling

We evaluate two negotiation-centric tasks: 1) bandwidth allocation; and 2) multiresource scheduling over a fixed planning horizon (e.g., one week). In each task, agents represent services with distinct minimum requirements and priorities and must negotiate an allocation that satisfies individual constraints under a global capacity constraint.

F. Robot Warehouse Coordination Task

In this task, a team of four specialist agents (picker, packer, transporter, and inventory manager), coordinated by the Leader and SSA, must jointly plan the fulfilment of a fixed set of warehouse orders under spatial and resource constraints. The environment is a 5×5 grid (Tables XV–XVI) containing shelves (S) with items 1–10, aisles (A), blocked cells (B), trolleys (T), and charging stations (C). Two aisles are blocked (cells marked B), one trolley is missing, and the Transporter has a low-battery constraint.

TABLE III
COMPOSITE SCORE AND WALL-CLOCK TIME OF CREWAI SEQUENTIAL AND COSMIC ACROSS A DIVERSE SET OF TASKS

Task	CrewAI Sequential		COSMIC	
	Time	Score	Time	Score
Math. Problems	10 m	0(Fail)	197 ± 39 s	100 ± 0.0
Multilingual Trans.	3 m 20 s	51	31 ± 8 s	95.0 ± 1.6
Short Story Gen.	7 m 2 s	85	289 ± 29 s	94.4 ± 1.5
Healthcare	4 m 36 s	75	101 ± 20 s	95.0 ± 1.9
Bandwidth Alloc.	2 m 7 s	85	95 ± 19 s	94.8 ± 2.2
Resource Sched.	1 m 38 s	85	123 ± 11 s	92.8 ± 1.9

Success is evaluated using the same composite score as in Section VI, instantiated as: 1) order fulfilment and constraint satisfaction (all requested items collected and delivered without violating blocked cells or battery limits); 2) respect for priority ordering (urgent $>$ express $>$ standard); 3) temporal efficiency (time to completion); and 4) collaboration quality among agents. Order item locations and obstacle positions are fully specified in Appendix C-G to enable reproduction.

Baseline reliability and failure modes: We quantify reliability using hard-constraint validity criteria: delivery completeness (all requested items delivered to the correct dispatch zones), blocked-aisle avoidance, battery recharging compliance, and correct priority ordering (Urgent $>$ Express $>$ Standard), all within the fixed interaction budget. Across $N = 5$ trials, COSMIC achieves a 100% validity rate (5/5). In contrast, all baseline frameworks achieve 0% validity (0/5) under these criteria, with dominant failures including attempts to traverse blocked cells, omission of urgent orders, and transporter battery depletion without recharging. A per-framework breakdown is reported in Appendix B (Table XIV).

We treat this task primarily as a stress test of COSMIC's coordination capabilities. Since baseline frameworks produced no valid solutions under the hard constraints (0/5), we report COSMIC's quantitative score and runtime for this task, and restrict baseline reporting to validity rates and qualitative failure patterns.

G. Comparison with Other Frameworks

1) Comparison With CrewAI Sequential: CrewAI Sequential arranges agents in a linear pipeline, which can work well when tasks have a strictly step-by-step flow. However, it struggles when multiple agents need frequent back-and-forth communication. Table III compares both frameworks on the six tasks.

a) Issues in Sequential Flow:

1) *Difficulty with consensus:* Agents downstream must inherit prior outputs, limiting direct negotiations or corrections.
2) *Delay in correction:* If an upstream agent makes an error, reruns often cascade through the entire chain.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE IV
COMPOSITE SCORE AND WALL-CLOCK TIME OF CREWAI HIERARCHICAL AND COSMIC ACROSS A DIVERSE SET OF TASKS

Task	CrewAI Hierarch		COSMIC	
	Time	Score	Time	Score
Math. Problems	8 m 33.1 s	0(Fail)	197 ± 39 s	100 ± 0.0
Multilingual Trans.	3 m 22.6 s	95	31 ± 8 s	95.0 ± 1.6
Short Story Gen.	8 m 3 s	88	289 ± 29 s	94.4 ± 1.5
Healthcare	5 m 8 s	60	101 ± 20 s	95.0 ± 1.9
Bandwidth Alloc.	8 m 4 s	88	95 ± 19 s	94.8 ± 2.2
Resource Sched.	7 m 18 s	60	123 ± 11 s	92.8 ± 1.9

TABLE V
COMPOSITE SCORE AND WALL-CLOCK TIME OF **AUTOGEN + MINIMAL COORDINATOR** AND **COSMIC** ACROSS TASKS

Task	AutoGen + Coord.		COSMIC	
	Time	Score	Time	Score
Math. Problems	12 m	0(Fail)	197 ± 39 s	100 ± 0.0
Multilingual Trans.	2 m 14 s	52	31 ± 8 s	95.0 ± 1.6
Short Story Gen.	6 m 31.2 s	78	289 ± 29 s	94.4 ± 1.5
Healthcare	3 m 43.5 s	90	101 ± 20 s	95.0 ± 1.9
Bandwidth Alloc.	8 m 6 s	60	95 ± 19 s	94.8 ± 2.2
Resource Sched.	4 m 3 s	88	123 ± 11 s	92.8 ± 1.9

Note: Baseline values are the best validated score among $N = 5$ trials; COSMIC is mean $\pm$ std over $K = 5$ runs.

TABLE VI
COMPOSITE SCORE AND WALL-CLOCK TIME OF **AUTOGEN (STANDARD)** AND **COSMIC** ACROSS TASKS

Task	AutoGen (standard)		COSMIC	
	Time	Score	Time	Score
Math. Problems	7 m 41 s	0(Fail)	197 ± 39 s	100 ± 0.0
Multilingual Trans.	4 m 32 s	50	31 ± 8 s	95.0 ± 1.6
Short Story Gen.	1 m 10 s	66	289 ± 29 s	94.4 ± 1.5
Healthcare	5 m 3 s	75	101 ± 20 s	95.0 ± 1.9
Bandwidth Alloc.	10 m 6 s	64	95 ± 19 s	94.8 ± 2.2
Resource Sched.	3 m 17 s	85	123 ± 11 s	92.8 ± 1.9

Note: AutoGen (standard) is the leaderless GroupChat configuration. Baseline values are the best validated score among $N = 5$ trials; COSMIC is mean $\pm$ std over $K = 5$ runs.

3) *Limited parallelism:* In tasks like resource scheduling or translation, multiple subtasks need simultaneous refinement, which the sequential pipeline cannot readily accommodate.

b) How COSMIC addresses these gaps: COSMIC's shared memory buffer and leader-driven approach mitigate these pitfalls.

1) *Shared context:* All agents see a rolling log of the conversation, allowing instant collaboration and quick correction of mistakes.
2) *Flexible turn-taking:* A specialized SSA ensures the right agent speaks next, bypassing strict top-to-bottom constraints.

2) Comparison With CrewAI Hierarchical:

a) Limitations in Rigidity and Communication:

1) *Bottleneck at the top:* The manager agent can become a single point of failure for communication. Slower tasks or repeated clarifications must pass through it, increasing latency.
2) *Reduced flexibility:* If agents need peer-to-peer coordination, hierarchical lines can slow responses and complicate direct interactions.
3) *Emergent behaviors restricted:* Complex tasks like negotiation or creative ideation rely heavily on spontaneous back-and-forth; a strict hierarchy curtails that.

b) How COSMIC Improves Collaboration:

1) *Leader as coordinator, not gatekeeper:* In COSMIC, the leader oversees final decisions but does not micromanage every exchange. This prevents bottlenecks and speeds up negotiation-heavy tasks.
2) *Shared memory for transparency:* Each agent sees other agents' contributions in near real-time, allowing rich two-way collaboration without passing everything through the manager.
3) *Speed and accuracy gains:* Table IV highlights that in tasks requiring emergent cooperation (e.g., Resource Scheduling), COSMIC outperforms hierarchical setups in both time and score.

3) Comparison With AutoGen: AutoGen is evaluated in two configurations (Table V and Table VI): 1) *AutoGen (standard)*, the default leaderless GroupChat; and 2) an auxiliary strengthened variant, *AutoGen + Minimal Coordinator*, which adds a lightweight coordination role to encourage turn discipline and trigger reconciliation when contradictions persist. Both variants are run under identical prompts, interaction budgets, and decoding settings (Section VI). For each variant, we report the *best validated composite score* among $N = 5$ trials as an optimistic upper bound, and we report validity rates separately (Appendix B). COSMIC results are reported as mean $\pm$ standard deviation over $K = 5$ runs.

a) Limitations Observed:

1) *Unstable turn-taking and speaker imbalance:* In AutoGen (standard), the absence of an external speaker arbiter can lead to over-participation by some agents and under-utilization of others, particularly in high-coupling tasks.
2) *Weak convergence and termination behavior:* Under a fixed interaction budget, AutoGen (standard) exhibits frequent nonconvergence or early termination with unresolved inconsistencies; this is reflected in reduced validity rates (Appendix B).
3) *No explicit shared rolling buffer:* AutoGen does not provide an explicit rolling shared-memory buffer in the sense used by COSMIC; as a result, important constraints and intermediate commitments can be inconsistently preserved across agents, increasing contradiction risk.

b) How COSMIC Resolves These Issues:

1) *SSA:* COSMIC enforces structured turn-taking (anti-hijacking and fairness), ensuring that relevant specialists are invoked at the right time and preventing domination/self-loop patterns.
2) *Shared rolling memory:* A single shared buffer keeps agents synchronized on the most recent commitments, reducing redundant work and contradiction.
3) *Leader-guided convergence:* COSMIC's Leader monitors coverage and consistency and terminates only when completion and stability conditions hold, reducing drift and avoiding premature termination.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE VII
COMPOSITE SCORE AND WALL-CLOCK TIME OF LANGCHAIN AND COSMIC ACROSS A DIVERSE SET OF TASKS

Task	LangChain		COSMIC	
	Time	Score	Time	Score
Math. Problems	20 m 52 s	0	197 ± 39 s	100 ± 0.0
Multilingual Trans.	1 m 2 s	88	31 ± 8 s	95.0 ± 1.6
Short Story Gen.	14 m 32 s	45	289 ± 29 s	94.4 ± 1.5
Healthcare	1 m 20 s	80	101 ± 20 s	95.0 ± 1.9
Bandwidth Alloc.	33 s	35	95 ± 19 s	94.8 ± 2.2
Resource Sched.	24 m 43 s	60	123 ± 11 s	92.8 ± 1.9

TABLE VIII
COMPARISON OF COSMIC AND MAD ON GSM8K

Framework	Accuracy on GSM8K	Topology/Variant
MAD (Fully Connected)	84.5	D = 1
MAD (Sparse)	86 ± 1.5	Best of D = 2/5, 3/5, 4/5
COSMIC	94 ± 1	NA

4) Comparison With LangChain: LangChain offers modular tools for chaining language models and external APIs in structured pipelines. While highly effective for tool integration and agent task routing, its multiagent coordination capabilities exhibit several limitations in collaborative problem-solving contexts. Table VII presents a direct comparison of LangChain and COSMIC across six benchmark tasks.

a) Limitations Observed: While LangChain supports custom agent definitions and execution flows, its default design often leads to incomplete or unreliable convergence in multiagent scenarios.

1) *Abrupt termination:* Agents frequently fail to converge on a final solution. Instead, the entire task process terminates without reaching consensus or resolution.
2) *Time inefficiency:* Task runtimes were significantly higher for complex challenges (e.g., 24 min for resource scheduling and over 20 min for math problems), making it unsuitable for time-sensitive coordination tasks.
3) *Inconsistent agent participation:* In many runs, only a subset of defined agents actively contributed to the solution. This lack of holistic agent involvement undermined result quality.
4) *Low and uneven output quality:* Output consistency varied significantly across tasks, with some (e.g., story writing) producing underdeveloped or disjointed results despite long runtimes.

b) How COSMIC addresses these issues: COSMIC resolves these limitations through its structured, leader-driven approach and shared memory model.

1) *Guided convergence:* With a dedicated leader and SSA, COSMIC ensures collaborative focus and guides agent discussions toward meaningful closure.
2) *Uniform agent activation:* Every agent receives an opportunity to contribute based on task demands and their designated roles, improving quality and robustness.
3) *Shared context and shorter times:* Real-time context sharing reduces repetition and speeds up consensus, resulting in faster convergence with higher solution accuracy.

As illustrated in Table VII, COSMIC outperforms LangChain across all tasks in both time and accuracy, highlighting the importance of structured, collaborative coordination mechanisms in multiagent frameworks.

H. Comparison With Google's MAD: The MAD framework, particularly in its fully-connected and sparse variants, has shown promise in enhancing reasoning accuracy via agent-led discussions and majority voting. However, when benchmarked on the GSM8K dataset—a grade school math reasoning benchmark. COSMIC exhibited a higher maximum score of 95, outperforming the best MAD variant, which achieved a score of 88. Detailed GSM8K results are summarized in Table III.

The MAD architecture initiates with agents generating individual solutions, followed by multiround debates where agents revise their responses based on peer feedback. This process is guided either through fully connected or sparsely connected topologies. Despite being innovative in balancing quality and computational cost, MAD still presents several limitations when compared with COSMIC.

1) Limitations of MAD:

a) *Convergence ambiguity:* MAD often relies on majority voting postdebate rounds, which can sometimes result in inconsistent or incorrect consensus, especially when incorrect agents dominate early discussions.
b) *Communication cost:* Even with sparse topologies, MAD incurs overhead due to multiround debate cycles and the need to retain previous peer solutions.
c) *Input sensitivity:* MAD's performance fluctuates based on the quality of initial agent outputs, as debate rounds start from those baselines.
d) *No central control:* The absence of a leader or priority agent can lead to stagnation or indecision, particularly in cases where agents disagree and voting yields a tie.

2) How COSMIC Outperforms MAD: COSMIC introduces several innovations that address the weaknesses observed in MAD.

a) *Leader-driven convergence:* COSMIC's leader agent not only resolves conflicts but also actively guides the coordination process toward final solution convergence.
b) *Structured turn management:* With a dedicated SSA, COSMIC avoids chaotic simultaneous responses and ensures the most relevant agent speaks when needed.
c) *Shared context for stability:* Unlike MAD, which may suffer from fragmented memory among agents, COSMIC's shared buffer ensures all agents work from a synchronized understanding of the problem and peer contributions.

While MAD's sparse topology variants offer substantial cost savings and improved debate efficiency over fully connected setups, COSMIC demonstrates that well-orchestrated coordination rather than just communication sparsity is crucial for solving reasoning-intensive tasks such as GSM8K with higher accuracy and consistency.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE IX
COMPOSITE SCORE AND WALL-CLOCK TIME OF OPENAI SWARM AND COSMIC ACROSS A DIVERSE SET OF TASKS

Task	OpenAI Swarm		COSMIC	
	Time	Score	Time	Score
Math. Problems	2 m 21 s	0	197 ± 39 s	100 ± 0.0
Multilingual Trans.	41 s	90	31 ± 8 s	95.0 ± 1.6
Short Story Gen.	75 s	30	289 ± 29 s	94.4 ± 1.5
Healthcare	55 s	90	101 ± 20 s	95.0 ± 1.9
Bandwidth Alloc.	41 s	40	95 ± 19 s	94.8 ± 2.2
Resource Sched.	51 s	70	123 ± 11 s	92.8 ± 1.9

I. Comparison With OpenAI Swarm

At the time of evaluation, OpenAI Swarm was still in an early experimental stage, primarily introduced as a developer sandbox for exploring multiagent orchestration. Despite its nascent maturity, we included Swarm in our benchmarking due to its emphasis on agent autonomy and decentralized collaboration—attributes closely aligned with our core objectives. We were particularly interested in testing how Swarm handled tasks requiring spontaneous cooperation without a rigid communication structure. Table IX summarizes the performance of Swarm against COSMIC across six collaborative task domains.

1) Observations on Swarm Performance: Given Swarm's experimental nature, its performance demonstrated both promising potential and critical limitations.

a) *Encouraging agent autonomy:* Swarm's agent design encourages flexibility, allowing agents to operate independently with clear role specializations and dynamic handoffs. This structure theoretically supports scalable collaboration.

b) *Rapid execution with inconsistent outcomes:* Swarm completed tasks quickly due to lightweight orchestration. However, quality varied widely, as shown by strong results in translation (score 90) and healthcare (90), but notably poor results in story generation (30) and math (0).

c) *Lack of persistent state:* Swarm's stateless design required external state tracking, complicating long-horizon tasks such as mathematics problem solving or story composition. The inability to maintain shared memory likely contributed to its failures on such tasks.

d) *No central decision-making:* Unlike COSMIC's leader-driven structure, Swarm lacks a unified agent to arbitrate decisions or guide coordination, resulting in uneven contributions and inconsistent convergence.

2) How COSMIC Improves on Swarm: While Swarm provides a sandbox to test novel agent coordination paradigms, COSMIC introduces architectural refinements for practical and reliable multiagent collaboration.

a) *Leader-guided execution:* COSMIC introduces a dedicated leader agent and Speaker Selector to manage conversation flow, enabling clear convergence on decisions and reducing agent collision.

b) *Shared memory context:* COSMIC maintains a rolling buffer shared across all agents, ensuring continuity and coherence even in long-running or interdependent tasks.

c) *Robust solution quality:* As seen in Table IX, COSMIC consistently outperformed Swarm in solution accuracy, particularly in tasks where the quality of output required sustained negotiation and memory management.

OpenAI Swarm remains a promising foundational framework for decentralized AI collaboration. However, its lack of convergence mechanisms, persistent state, and structured coordination made it less effective on coordination-heavy tasks compared with COSMIC.

In sum, the benchmarking results clearly indicate that our leader-driven, context-rich framework provides meaningful advantages over conventional approaches in tasks requiring varying degrees of collaboration and coordination.

J. Ablation Studies of COSMIC Components

To disentangle the contributions of COSMIC's core modules, we conducted an ablation study on the *resource scheduling* task (described in Section VIII). We select resource scheduling because it is a high-coupling negotiation setting that reliably exposes coordination failure modes (non-convergence, constraint omission, and speaker imbalance) observed in other frameworks. In this setting, a team of four specialist agents (A, B, C, D), each with private constraints over a multi-day resource schedule, must negotiate and reach a consensus on a joint allocation that respects both individual requirements and a global capacity constraint.

We compare the full COSMIC configuration against three ablated variants.

1) *No-Leader*: Removes leader-driven task tracking and termination; specialists negotiate directly without a central coordinator.

2) *No-SSA*: Disables structured speaker selection and anti-hijacking; agents are allowed to self-invoke freely.

3) *No-Memory*: Restricts agents to conditioning only on the last message (no persistent shared buffer), mimicking a purely local, myopic negotiation.

All variants use the same underlying LLM and agent prompts. We run $K = 5$ independent trials per setting. A run is considered valid if it terminates within the interaction budget and satisfies all hard constraints specified in the task prompt (agent minima and global capacity).

Table X summarises the composite performance score (Section VI), wall-clock time, and number of messages.

1) Impact of the Leader Agent: Removing the Leader Agent leads to a substantial drop in solution quality (from 93.8 to 67.3 average score, a decrease of 26.5 points), accompanied by a noticeable increase in variance ($\sigma = 12.8$). In the *No-Leader* variant, no single agent is responsible for tracking global progress, enforcing the termination predicate, or ensuring that all parties' private constraints are jointly satisfied. Manual inspection of the negotiation logs indicates that agreements often appear locally reasonable from the perspective of the agents who dominate the dialogue, but silently violate one or more outstanding constraints (e.g., an agent's daily minimum allocation is never addressed). Although the mean wall-clock

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE X
ABLATION STUDY OF COSMIC COMPONENTS ON THE COORDINATION-CENTRIC RESOURCE SCHEDULING TASK ($K = 5$ RUNS PER VARIANT)

Variant	Score	Time (s)	Messages
Full COSMIC	93.8 ± 3.0	48.3 ± 10.2	9.8 ± 1.1
No-Leader	67.3 ± 12.8	29.3 ± 5.6	7.0 ± 1.2
No-SSA	77.4 ± 11.3	61.0 ± 19.2	12.4 ± 3.6
No-Memory	43.4 ± 43.1	44.0 ± 19.0	5.5 ± 3.1

Note: Removing any of the three core modules—Leader, SSA, or Shared Memory—significantly degrades performance, and each affects a different aspect of quality vs. efficiency.

time decreases (from 48.3 to 29.3 s) due to the absence of periodic leader heartbeats, this speed-up is achieved by converging prematurely to incomplete or inconsistent schedules. This confirms that leader-driven coordination is central to COSMIC's ability to reliably converge on a high-quality joint solution that respects all stakeholders.

2) Impact of the SSA: Disabling the SSA degrades performance from 93.8 to 77.4 (a drop of 16.4 points) and increases both the average number of messages (from 9.8 to 12.4) and the wall-clock time (from 48.3 to 61.0 s). In this *No-SSA* setting, we frequently observe *self-invocation loops*: one or two specialists repeatedly call themselves, even after having largely stated their case, while others rarely get the floor or are never invoked at all. As a result, some agents' constraints remain unvoiced or are raised too late to influence the emerging schedule. This behavior is consistent with the higher message counts and longer runtimes of the No-SSA variant, which expend additional turns revisiting the same perspectives without materially improving the final allocation. The structured turn-taking rules of the SSA (heartbeat scheduling, antihijacking, and fairness) therefore, play a dual role: they improve solution quality and also prevent pathological dialogue patterns that waste tokens and time.

3) Impact of the Shared Memory Buffer: The *No-Memory* variant exhibits the most severe degradation, with the average score dropping to 43.4 and the standard deviation increasing dramatically ($\sigma = 43.1$). This high variance reflects unstable behavior: some runs manage to converge to a partially acceptable allocation, while others fail validity criteria by drifting away from the original problem, forgetting earlier constraints, or losing track of which agent has accepted which clauses. Without the rolling shared buffer, each agent's view of the negotiation effectively collapses to the last utterance, making it difficult to maintain a globally coherent, multiday schedule. The wall-clock time and number of messages are somewhat lower on average (since conversations often terminate quickly after going off-track), but this is due to early failure rather than efficient convergence. These results empirically confirm that the shared memory buffer is crucial for preserving long-horizon semantic consistency in multiagent negotiations.

4) Turn Distribution and Interaction Patterns: Beyond aggregate scores and timings, the ablation logs reveal distinct interaction patterns. In the full COSMIC configuration, specialist turn counts are relatively balanced: on average, each of the four specialists contributes between 1 and 3 turns per episode, and the Leader is invoked around 3.8 times per run, providing regular global progress checks. No single agent systematically dominates the conversation, and all parties have opportunities to present and negotiate their constraints.

TABLE XI
TURN DISTRIBUTION FOR AUTOGEN GROUPCHAT ON THE RESOURCE SCHEDULING TASK (TWO REPRESENTATIVE EPISODES)

Episode	Leader	Agent A	Agent B	Agent C	Agent D
1	4 (17%)	2 (8%)	0 (0%)	14 (58%)	4 (17%)
2	3 (12%)	7 (27%)	10 (38%)	4 (15%)	2 (8%)

Note: Values are counts, with percentages of total turns in parentheses.

In the *No-SSA* variant, several runs display extreme speaker imbalance: a single specialist can account for roughly two-thirds of all nonleader turns, while another specialist may never speak. This imbalance explains why some constraints remain unaddressed despite longer dialogues. The *no-leader* runs, by contrast, feature more balanced specialist counts but frequent failures to integrate their proposals into a globally consistent schedule, consistent with the lower scores despite moderate message counts. Finally, in the *no-memory* setting, many episodes terminate early with few specialist turns; the resulting allocations often omit previously stated constraints or even diverge from the original task description. Taken together, these patterns align with the quantitative instability observed for the ablated variants and further support the role of the shared buffer, SSA, and Leader in stabilising negotiation dynamics.

K. Qualitative Failure Analysis

To better understand the performance gaps, we analyzed the interaction logs of the baseline frameworks to identify specific behavioral failure modes.

1) Legacy Group-Chat (AutoGen): We observed two primary failure patterns in leaderless group chats. First, the *lack of authoritative termination* led to solution deterioration. In our leaderless GroupChat setup, termination was not centrally governed, and conversations were often ended prematurely by an overconfident agent emitting a stop signal before contradictions were resolved. Conversely, when agents failed to terminate, the conversation suffered from *semantic drift*: after reaching a valid solution, agents continued to generate tokens, eventually introducing hallucinations or irrelevant details that degraded the final score. Second, without a strong Speaker Selector, we observed frequent *self-invocation loops*, where a single agent monopolized the context window, preventing specialists from correcting errors.

2) Speaker Imbalance in AutoGen GroupChat: To quantitatively substantiate the speaker-imbalance issue in AutoGen, we analysed full dialogue logs for the resource scheduling task. Table XI reports the number of turns taken by each agent in two representative episodes.

3) Sequential and Hierarchical Pipelines (CrewAI): In the *sequential* setting, we observed *knowledge transfer failures*. While successor agents acknowledged their turn, they

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

frequently failed to build upon the predecessor's output, instead attempting to solve the entire problem statement from scratch independently. In the Mathematics task, this resulted in five agents solving the problem five separate times (often incorrectly) rather than decomposing it.

In the *hierarchical* setting, the failure mode was *orchestration thrashing*. The manager agent often invoked subordinates in a nonlogical order (e.g., calling a final-step verifier before the initial solver). Furthermore, the manager frequently interrupted agents mid-task, leading to incomplete outputs. The final answer was often a lossy summary generated by the manager itself, which discarded valid details produced by the workers.

4) Stateless Coordination (OpenAI Swarm): Swarm's failure mode stemmed from its *ephemeral context*. Since agents are instantiated as functions without a persistent shared memory, they lacked a unified view of the problem history. While this architecture resulted in very low latency (time score $\approx$ 90–100), it prevented the deep reasoning chains required for the Story Generation and Math tasks, resulting in disjointed narratives and inability to backtrack on calculation errors.

5) Voting Deadlocks (MAD): While the MAD framework excels at consensus formation on many problems, our analysis suggests it can enter *stagnation states* when agents remain split across competing solutions and no designated controller exists to enforce resolution. Unlike COSMIC, which uses a Leader to force a decision based on a consistency score $H(t)$, debate-only frameworks struggle to resolve legitimate ambiguity without external intervention.

IX. Testing Pure Collaboration

Although several benchmark tasks involve coordination, some can still be solved with limited interaction by reducing the problem to loosely coupled subtasks. To isolate settings where sustained coordination is necessary, we include a warehouse robot coordination stress test in which specialists must jointly satisfy spatial, resource, and priority constraints (Section VIII-F).

In this setting, agents assume specialized roles (Picker, Packer, Transporter, Inventory Manager) and must coordinate over a shared 5×5 layout with blocked cells, limited trolley availability, and a transporter battery constraint. The team must fulfil four orders while respecting priority ordering (Urgent $>$ Express $>$ Standard) and feasibility constraints (e.g., blocked-aisle avoidance and recharging rules).

Across repeated trials, COSMIC produced constraint-consistent plans in this setting: agents explicitly referenced shared state (layout, obstacles, trolley status, and battery level), revised allocations when conflicts were identified, and maintained priority ordering under the Leader's periodic checks. In contrast, under the same hard-constraint validity criteria and interaction budget, the evaluated baselines produced no valid solutions on this stress test (Section VIII-F). These results support the view that explicit turn arbitration and shared state are particularly beneficial in tightly coupled, multiconstraint coordination tasks.

X. Limitations and Future Work

While COSMIC demonstrates strong performance on coordination-intensive tasks, our analysis also reveals several limitations related to scalability, fault tolerance, and computational overhead. In this section, we discuss these trade-offs and outline a roadmap for future architectural enhancements.

A. Centralization and Scalability Bottlenecks

As shown in our team-size experiments (Section VII), performance tends to peak at approximately five agents before exhibiting diminishing returns. Beyond this regime, the *Leader Agent* can become a communication bottleneck: the Leader must repeatedly read the shared memory, integrate contributions, and update task status at each heartbeat. As team size and conversation length grow, the Leader's per-heartbeat processing cost increases with both the number of agents (more candidate contributions to reconcile) and the amount of recent dialogue retained in M, increasing latency and the risk of context overload.

In scenarios requiring dozens of agents (e.g., large-scale simulations or domain-diverse expert collectives), a single Leader may therefore be insufficient to coordinate all interactions efficiently.

1) Future Roadmap: A promising direction is to explore a *hierarchical or "fractal" leadership architecture*. In such a design, the overall team would be partitioned into small subsquads (e.g., groups of four to six agents), each overseen by a local squad leader. These squad leaders would in turn report to a global leader responsible for high-level task planning and final aggregation. Conceptually, this distributes coordination load from a single central node to a shallow leadership hierarchy, potentially reducing the effective coordination complexity per leader and improving scalability to larger teams.

B. Single Point of Failure (SPOF)

The same centralisation that improves coordination also introduces a clear single point of failure. If the Leader becomes unresponsive due to API timeouts, repeated inconsistencies, or misalignment with the original task (e.g., misinterpreting the termination criterion), the current version of COSMIC has no built-in mechanism to recover. In such cases, the conversation may stall or terminate with an incorrect solution, since no other component is authorised to override or replace the Leader.

1) Future Roadmap: To improve fault tolerance, we plan to investigate a *dynamic leader election protocol* inspired by classical distributed-systems leader election and failover mechanisms (e.g., quorum-based replacement). Concretely, the SSA and specialist agents could monitor the Leader's behavior for signs of failure (e.g., repeated incoherent summaries, violation of the stability conditions in the termination predicate, or prolonged absence of heartbeats). If such criteria are met, the agents could trigger a lightweight election process in which one of the specialists is temporarily promoted to a new Leader role, inheriting the shared memory and task brief. This would allow

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

the system to recover from Leader failures rather than treating them as terminal.

C. Overhead in Low-Coupling Tasks

Our results highlight that COSMIC is particularly effective for high-coupling, high-cohesion tasks where agents must repeatedly condition on each other's outputs. However, the same design introduces unnecessary overhead for simple or loosely coupled tasks. The shared-memory buffer, periodic leader heartbeats, and SSA-based turn-taking incur additional messages and tokens compared with minimal pipelines or stateless routing.

For highly independent, parallelisable workloads (e.g., "generate 50 unrelated haikus" or batch translation with no cross-example interaction), this overhead is difficult to justify: a simple parallel executor or a lightweight routing scheme (such as OpenAI Swarm or basic LangChain chains) is likely to be faster with negligible quality loss.

1) Future Roadmap: One natural extension is an *adaptive topology router*. This meta-controller would estimate a task's coupling and cohesion (as formalised in Section IV) before execution. Low-coupling tasks would be routed to a lightweight, largely parallel engine (e.g., a Swarm-like configuration or simple chains), while high-coupling tasks would instantiate the full COSMIC pipeline with Leader, SSA, and shared memory. In principle, this would yield the best of both worlds: minimal overhead for simple tasks and richer coordination for complex, tightly coupled problems.

D. Static Memory Window Constraints

In the current implementation, the shared memory window size N is a fixed hyperparameter tuned per task. This task-specific tuning is effective in practice but also brittle: a complex, long-horizon reasoning problem might require $N \approx 20$ to retain key steps, whereas a short translation task may only need $N \approx 5$. If N is set too small, critical context may be dropped; if it is set too large, prompts grow unnecessarily and increase latency and token costs.

1) Future Roadmap: A more flexible approach is to replace the fixed rolling window with a *semantic attention buffer*. Instead of discarding messages purely based on recency, the memory module could maintain a larger pool of past messages and select the N most relevant ones at each turn based on embedding similarity or learned importance scores. This would allow the effective context window to adapt dynamically to the reasoning requirements of the current step (e.g., preserving early constraints or definitions that remain important later), while still enforcing a hard bound on the number of tokens passed to the underlying LLM.

XI. Results and Conclusion

Across six coordination-intensive tasks: 1) mathematical reasoning; 2) multilingual translation; 3) story generation; 4) healthcare diagnostics; 5) bandwidth allocation; and 6) resource scheduling. COSMIC achieves consistently strong solution quality and reliable convergence under a fixed interaction budget (Figs. 5 and 6).

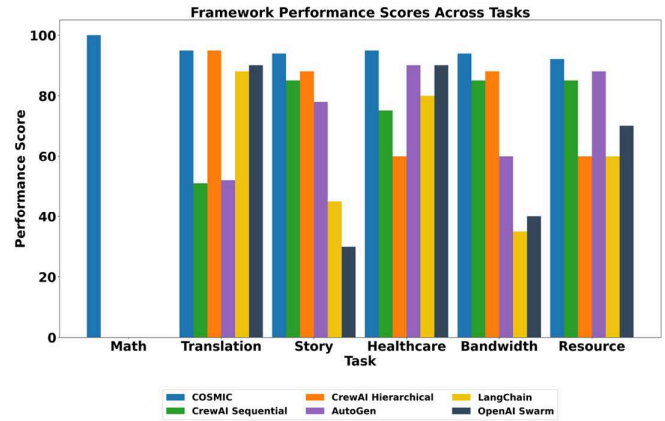

Fig. 5. Composite performance scores of evaluated frameworks across tasks. COSMIC values are mean ± std over $K = 5$ runs; baselines are best validated scores among $N = 5$ trials per task (Section VI).

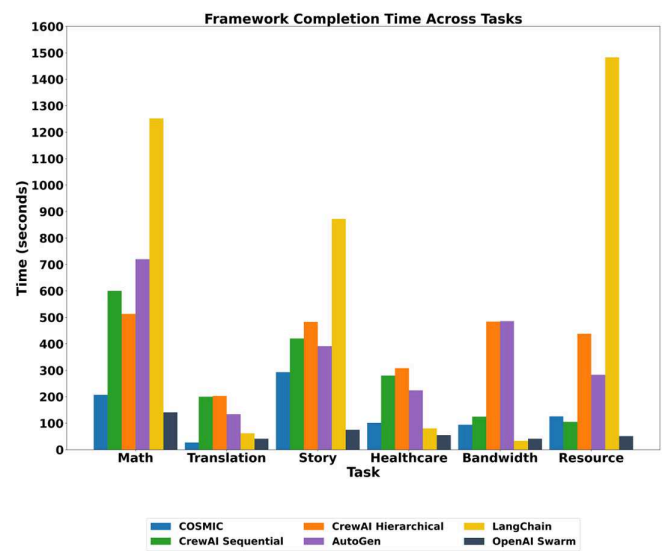

Fig. 6. Mean wall-clock completion time across tasks (lower is better). COSMIC values are mean ± std over $K = 5$ runs; baselines are mean over $N = 5$ trials.

Averaging protocol: For the *average performance-score improvement* numbers reported below, we exclude the mathematics task from score averaging because all baseline frameworks produced zero valid runs on that task (0/5), yielding no validated score to average against. We *do* retain the mathematics task in time reporting to reflect compute overhead under budget exhaustion and non-convergence behavior.

Score improvements and runtime: Relative to CrewAI Hierarchical and CrewAI Sequential, COSMIC achieves average composite-score improvements of 20.2% and 23.3%, while reducing mean wall-clock time by 57.3% and 50.8%, respectively. Against AutoGen, COSMIC improves average performance by

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE XII
VALIDITY RATES UNDER TASK-SPECIFIC HARD-CONSTRAINT CRITERIA AND A FIXED INTERACTION BUDGET

Framework	Math	Translation	Story	Healthcare	Bandwidth	Resource Sched.
COSMIC	5/5	5/5	5/5	5/5	5/5	5/5
CrewAI Sequential	0/5	4/5	4/5	3/5	4/5	5/5
CrewAI Hierarchical	0/5	2/5	4/5	3/5	4/5	4/5
AutoGen (Standard)	0/5	2/5	3/5	2/5	2/5	1/5
AutoGen + Minimal Coordinator	0/5	4/5	4/5	5/5	4/5	3/5
LangChain	0/5	3/5	2/5	3/5	4/5	2/5
OpenAI Swarm	0/5	3/5	4/5	4/5	4/5	4/5

Note: Each cell reports valid runs / N ($N = 5$ trials per framework per task).

38.9% versus the standard leaderless GroupChat configuration, and by 27.7% versus the strengthened *AutoGen + Minimal Coordinator* variant, while also reducing mean runtime by 55.9% and 61.4%, respectively. Against LangChain, COSMIC yields a 52.6% improvement in average score and a 77.6% reduction in runtime. Compared with OpenAI Swarm, COSMIC attains a 46.9% higher average score but incurs higher runtime (109.9% slower), consistent with the additional coordination and shared-memory overhead required for higher-quality convergence on high-coupling tasks.

Reliability under hard constraints: Beyond mean scores, validity rates highlight robustness under hard-constraint criteria (Section VI). Under the standard leaderless AutoGen GroupChat setup, only 10 out of 30 total runs across the six tasks were valid (33.3%), whereas COSMIC achieved 30/30 valid runs (100%) under the same trial protocol (5 runs $\times$ 6 tasks). This reliability gap is most pronounced on high-coupling tasks, where leaderless turn-taking and fragmented state lead to nonconvergence or constraint violations.

GSM8K benchmark: On GSM8K, COSMIC attains $94 \pm 1\%$ accuracy, exceeding the reported performance of the sparse multiagent debate configuration ($86 \pm 1.5\%$) published by Google Research [6], indicating that leader-guided convergence with shared state can complement debate-style coordination on reasoning benchmarks.

In sum, COSMIC's combination of shared memory, explicit leader-driven termination, and SSA-based turn arbitration improves both convergence reliability and solution quality on coordination-heavy tasks, while maintaining competitive runtime on most evaluated workloads.

APPENDIX A
TERMINATION HYPERPARAMETER SENSITIVITY

To assess robustness of COSMIC's termination mechanism, we perform a one-factor-at-a-time sensitivity sweep around the default values ($\theta_H = 0.8$, $\varepsilon_C = \varepsilon_H = 0.1$, $T_{\text{leader}} = 7$) on the coordination-centric resource scheduling task. For each setting, we run $K = 5$ trials with identical prompts and decoding parameters and report composite score, message count, and failure count (invalid schedule or failure to terminate). Results are summarized in Table XIII.

TABLE XIII
SENSITIVITY SWEEP OF TERMINATION HYPERPARAMETERS (ONE-FACTOR-AT-A-TIME) ON THE RESOURCE SCHEDULING TASK ($K = 5$ RUNS PER SETTING)

Setting	Score	Messages	Failures
$\theta_H = 0.7$	91.9	8.7	0
$\theta_H = 0.8$ (default)	93.8	9.8	0
$\theta_H = 0.9$	94.5	11.4	0
$\varepsilon_C = \varepsilon_H = 0.05$	94.4	12.3	0
$\varepsilon_C = \varepsilon_H = 0.10$ (default)	93.8	9.8	0
$\varepsilon_C = \varepsilon_H = 0.20$	92.6	8.4	1
$T_{\text{leader}} = 5$	94.6	12.0	0
$T_{\text{leader}} = 7$ (default)	93.9	9.8	0
$T_{\text{leader}} = 9$	92.7	8.9	1

APPENDIX B
BASELINE VALIDITY RATES ACROSS TASKS

We report *validity rates* to quantify reliability under a fixed interaction budget (Table XII). A trial is considered *valid* if the framework: 1) terminates within the round budget; and 2) satisfies task-specific hard-constraint criteria (defined per task in Appendix C and the task rubrics). For baseline frameworks, the main-text score tables report the *best validated composite score* among the valid trials as an optimistic (baseline-favoring) upper bound, while this appendix reports validity rates across repeated trials.

A. Warehouse Task Validity (Stress Test)

For the warehouse coordination stress test, validity is defined by hard constraints: delivery completeness, blocked-aisle avoidance, battery recharging compliance, and correct priority ordering (Urgent > Express > Standard). Across $N = 5$ trials, COSMIC achieves a 100% validity rate (5/5), whereas all baseline frameworks achieve 0% validity (0/5) under these criteria (Table XIV).

Qualitative inspection of invalid baseline runs shows recurring failure patterns: 1) attempts to traverse blocked cells; 2) omission or misordering of urgent orders; 3) transporter battery depletion without routing to a charging station; and 4) incomplete or internally inconsistent plans.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

TABLE XIV
WAREHOUSE TASK VALIDITY RATES UNDER HARD-CONSTRAINT CRITERIA ($N = 5$ TRIALS)

Framework	Validity (%)
COSMIC	100 (5/5)
CrewAI Sequential	0 (0/5)
CrewAI Hierarchical	0 (0/5)
AutoGen (Standard)	0 (0/5)
AutoGen + Minimal Coordinator	0 (0/5)
LangChain	0 (0/5)
OpenAI Swarm	0 (0/5)

TABLE XV
WAREHOUSE LAYOUT (5×5 GRID)

S: Item1, Item2	A	S: Item3, Item4	T	A
A	B	A	A	C
S: Item5, Item6	A	S: Item7	B	A
C	A	T	A	S: Item8, Item9
A	S: Item10	A	A	A

APPENDIX C
TASK DESCRIPTION PROMPT

Each task was designed with a standardized problem statement to ensure uniformity across all evaluated frameworks. This problem statement was presented to the agents to clearly define the objective they needed to accomplish. The precise problem statement, along with the corresponding task name, is documented below for reference.

In the context of COSMIC, problem statements were introduced to all agents as an initial system message disseminated by the leader within a shared group communication channel. This message was subsequently stored in each agent's memory, ensuring a common knowledge base for all participants. Following the initialization, the leader dynamically assigned specialized roles to individual agents, facilitating a structured problem-solving workflow through collaborative discussions.

A. Multilingual Language Translation Task

Problem statement: The following multilingual message contains critical cybersecurity intelligence and requires immediate translation into English for dissemination to relevant stakeholders.

Die infiltradores haben planificado einen ataque auf das système informatique unserer infrastruktur crítica. Sie nutzen vulnerabilidades in der sécurité software aus, um acceso no autorizado zu erhalten und datos sensibles zu exfiltrieren. Es ist essentiel, dass wir actualicemos alle protocoles und contraseñas inmediatamente, um die protection zu erhöhen. Les utilisateurs müssen être vigilants gegenüber phishing-Versuchen und e-mails mit adjuntos sospechosos meiden. Se recomienda eine formation zur conscience de sécurité für alle empleados. Außerdem, sollten wir die authentification multi-facteurs implementieren und firewalls configurar. Collaboration internationale ist erforderlich, um die cyberattaque zu stoppen und partager renseignements über nouvelles menaces. Nuestros partenaires haben offres zur aide angeboten; por favor, contactez sie, um die ressourcen zu koordinieren. Zusätzlich, müssen wir plan de contingencia entwickeln, um opérations bei sistemas comprometidos aufrechtzuerhalten. Es crucial die logs zu analyser und anomalías zu identifizieren, die auf activité malveillante hinweisen. Juntos, können wir die amenaza neutralisieren und die integridad unserer datos proteger. Bitte informieren alle departamentos über die situación und instruiert sie, mejores prácticas zu seguir. La coopération zwischen équipes ist fondamentale für den succès unserer défense gegen diese attaque sophistiquée.

The translation must preserve contextual accuracy, especially in conveying critical cybersecurity terminologies. Agents must ensure that any idiomatic expressions are rendered with clarity in English while maintaining the original urgency and intent.

B. Mathematics Problem Task

Initial Message: Problem 1: Solve for N in the equation

$$2x^3 - 5x^2 + x - 7 = 0$$

where $x = \sqrt{N}$.

Problem 2: Identify prime numbers N such that both N and $N + 2$ are prime (twin primes) while ensuring $N \equiv 3 \mod 4$.

Problem 3: Solve for N in the combinatorial equation

$$\binom{N}{3} = 364.$$

Problem 4: Determine integer values of N that satisfy the integral equation

$$\int_0^{\pi} \sin(Nx)\, dx = 0.$$

Problem 5: Identify integer values of N such that a regular polygon with N sides has an internal angle that is an integer. Use the formula

$$\alpha = \left(1 - \frac{2}{N}\right) \times 180^\circ.$$

Agents must coordinate their approaches, ensuring that insights from one problem inform solutions to others. The leader will oversee the discussion and validate the results.

C. Resource Scheduling Task

Initial message: Leader: Assign tasks now. Agents A, B, C, D: Present your constraints. Start scheduling.

Each agent is assigned unique constraints and must negotiate an optimized scheduling solution while ensuring minimal resource conflicts. The objective is to evaluate negotiation efficiency, adaptability, and inter-agent coordination.

D. Story Writing Task

Initial message: Generate an adventure-thriller short story of approximately 2000 words. The narrative must include well-structured plot development, dynamic character interactions, and logical coherence.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

E. Bandwidth Allocation Task

Initial message: Leader: Agents A, B, C, D, report your QoS needs now. Negotiate bandwidth so all critical requirements are met.

Agents must determine an optimal bandwidth allocation strategy to meet the requirements of all connected systems while minimizing latency and ensuring fairness.

F. Medical Diagnosis Task

Initial message: Leader: Agents A-D, patient presents with chest pain, arrhythmia, dizziness, numbness, fatigue, weight changes, and cold sensitivity. Initial tests show mild arrhythmias, unclear neurological signs, and possible thyroid dysfunction. Work together to reach a conclusive diagnosis. Additional test results will be provided midway.

Midexecution update: The following test results were introduced mid-discussion to evaluate how agents handle evolving medical data: Test results - blood tests: Elevated TSH, low T3/T4 (suggesting hypothyroidism). Autoimmune markers: Positive for antithyroid peroxidase (anti-TPO) antibodies. Imaging (MRI of Brain): Subtle, nonspecific white matter lesions in the frontal lobe. No tumors or acute infarcts. Inflammatory markers: Mildly elevated ESR and CRP. Cardiac monitoring (holter): Occasional premature atrial contractions, no sustained dangerous arrhythmias. Body temperature: 37.8 °C (100.0 °F), indicating mild fever.

Agents must integrate new findings into their diagnostic process, reevaluate hypotheses, and propose an optimal treatment strategy.

G. Robot Warehouse Task

Initial message: Leader: Agents A, B, C, D, we face a complex scenario.

1) *Layout reference:* see warehouse table below.
2) *Orders:* see order details below.
3) *Issues:* Two blocked aisles, one trolley missing, C's battery is low.
   Propose initial solutions to address these issues and fulfil the orders efficiently.

For clarity, the environmental parameters are

1) *Grid:* 5×5, indexed from top-left to bottom-right.
2) Shelves and items
   a) Row 1, Col 1: S (Item1, Item2); Row 1, Col 3: S (Item3, Item4);
   b) Row 3, Col 1: S (Item5, Item6); Row 3, Col 3: S (Item7);
   c) Row 4, Col 5: S (Item8, Item9); Row 5, Col 2: S (Item10).
3) Obstacles and resources
   a) Blocked aisles at Row 2, Col 2 and Row 3, Col 4 (marked B),
   b) Trolleys at Row 1, Col 4 and Row 4, Col 3 (one trolley assumed unavailable),
   c) Charging stations at Row 2, Col 5 and Row 4, Col 1 (marked C),
   d) The Transporter starts with a low-battery constraint and must recharge at a C cell before extended use.
4) *Orders and priorities:* As listed in Table XVI, with priority levels (urgent, express, standard) and dispatch zones 1–3.

TABLE XVI
ORDER DETAILS

Order ID	Priority	Items
1	Urgent	Item1, Item5 (Dispatch Zone 1)
2	Express	Item3 (Dispatch Zone 2)
3	Standard	Item8, Item9 (Dispatch Zone 3)
4	Urgent	Item7 (Dispatch Zone 1)

The objective of the agents is to produce a coordinated plan that. 1) fulfils all orders; 2) honours the priority ordering (urgent first); 3) avoids blocked cells and respects trolley and battery constraints; and 4) minimises total completion time. These criteria feed into the composite performance score defined in Section VI.

H. Scaling Task

Initial message: Delegate the work to the summarizer agents (Agent0, Agent1, Agent2, ... Agent *n*) and work towards summarizing the novel Flatland - Flatland.txt

APPENDIX D
AGENTS AND PROMPTS USED IN TASKS

For all tasks used in comparing and benchmarking different agentic frameworks, the number of agents created and the roles they were assigned remained consistent to facilitate an equal testing environment for all. There is a slight modification in the prompting used for COSMIC to facilitate better communication between the agents. In COSMIC runs, any 'choose next speaker' instruction is treated as a nomination; the SSA makes the final selection.

The agents used along with their prompts for each task are mentioned below.

A. Mathematics Problem Task

This task involved six agents collaborating in a structured group chat to determine a valid integer value of N that satisfies all given mathematical constraints.

1) Algebra Specialist: Prompt: You are an expert in algebra, responsible for analyzing and solving algebraic equations. Solve only and only problem one. Share your reasoning and results in the group chat. Be open to adjusting conclusions based on input from others. Avoid unrelated discussions. Always choose the next speaker after your turn.

2) Number Theory Specialist: Prompt: You specialize in number theory, focusing on prime numbers and congruences. Solve only and only problem two. Determine potential values of N and share findings in the group chat. Adjust conclusions if needed based on team input. Avoid unrelated discussions. Always choose the next speaker after your turn.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

3) Combinatorics Specialist: Prompt: You are an expert in combinatorics, solving problems involving combinations and binomial coefficients. Solve only and only problem three. Find integer values of N that satisfy the problem. Share reasoning and results in the group chat. Be open to revising conclusions based on input from others. Avoid unrelated discussions. Always choose the next speaker after your turn.

4) Calculus Specialist: Prompt: As a calculus expert, your role is to analyze definite integrals. Solve only and only problem four. Determine possible values of N that satisfy the integral condition. Share reasoning and findings in the group chat. Be open to adjusting conclusions based on team feedback. Avoid unrelated discussions. Always choose the next speaker after your turn.

5) Geometry Specialist: Prompt: You focus on geometric properties, particularly angles and polygons. Solve only and only problem five. Determine integer values of N that satisfy the geometric condition. Share findings and reasoning in the group chat. Adjust conclusions if needed based on other agents' insights. Avoid unrelated discussions. Always choose the next speaker after your turn.

6) Leader: Prompt: You are the Team Leader in charge of synthesizing responses. You do not solve any mathematical problems but must wait for all five agents to provide their values of N. Provide the final answer only when all responses are received. If no integer solution exists, state: *"No integer value found"*. End the discussion with the keyword *TERMINATE*. Avoid introducing any new mathematical conditions. Always choose the next speaker after your turn.

B. Translation Task

This task involved four agents collaborating in a structured group chat to translate a multilingual message containing Spanish, French, and German words into a coherent English message. Each agent specialized in translating a single language, while the leader synthesized the final message.

1) Spanish Expert: Prompt: You are fluent in Spanish and responsible for translating only the Spanish words in a mixed-language message. Identify and accurately translate all Spanish words into English. Collaborate with other agents to assemble the full translation. Discuss ambiguities or alternative meanings when necessary. Share your translations in the group chat, indicating their position if possible. Do not attempt to translate words in other languages. Maintain confidentiality; do not reveal the original Spanish words. Wait for the Leader to synthesize the final message before concluding. Communicate only in English. Always choose the next speaker after your turn.

2) French Expert: Prompt: You are fluent in French and responsible for translating only the French words in a mixed-language message. Identify and accurately translate all French words into English. Collaborate with other agents to assemble the full translation. Discuss ambiguities or alternative meanings when necessary. Share your translations in the group chat, indicating their position if possible. Do not attempt to translate words in other languages. Maintain confidentiality; do not reveal the original French words. Wait for the Leader to synthesize the final message before concluding. Communicate only in English. Always choose the next speaker after your turn.

3) German Expert: Prompt: You are fluent in German and responsible for translating only the German words in a mixed-language message. Identify and accurately translate all German words into English. Collaborate with other agents to assemble the full translation. Discuss ambiguities or alternative meanings when necessary. Share your translations in the group chat, indicating their position if possible. Do not attempt to translate words in other languages. Wait for the Leader to synthesize the final message before concluding. Communicate only in English. Always choose the next speaker after your turn.

4) Leader: Prompt: You are the Team Leader overseeing the translation process. Monitor collaboration among agents and ensure accurate translations. Wait until all agents have shared their translations and resolved any ambiguities. Do NOT translate anything yourself. Synthesize the translations into a single, coherent English message. Ensure the message retains its original meaning and urgency. Do NOT include any content related to scheduling meetings or further discussions. Present the final message clearly and concisely. Include the keyword *TERMINATE* at the end of your final message to signal the end of the group chat. Always choose the next speaker after your turn.

C. Story Writing Task

This task involved four agents collaborating in a structured group chat to write a short story of exactly 2000 words. Each agent specialized in different aspects of storytelling, including character development, plot structure, world-building, and final story compilation.

1) Character Developer: Prompt: You are responsible for creating engaging and multidimensional main characters for a short story of 2000 words. You are in a group chat with the plot developer, world developer, and your Leader. Collaborate with them to write the story. After finalizing the initial elements, work towards finalizing the story. Stick to the word limit strictly. Do not schedule meetings or assume responses. Always choose the next speaker after your turn.

2) Setting and World Developer: Prompt: You are responsible for creating a rich and immersive setting for a short story of 2000 words. You are in a group chat with the character developer, plot developer, and your Leader. Collaborate with them to write the story. After finalizing the initial elements, work towards finalizing the story. Stick to the word limit strictly. Do not schedule meetings or assume responses. Always choose the next speaker after your turn.

3) Plot Developer: Prompt: You are responsible for developing the main plot for a short story of 2000 words. You are in a group chat with the character developer, world developer, and your Leader. Collaborate with them to write the story. After finalizing the initial elements, work towards finalizing the story. Stick to the word limit strictly. Do not schedule meetings

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

or assume responses. Always choose the next speaker after your turn.

4) Leader: Prompt: You are the head writer and leader overseeing the collaborative storytelling project. You are in a group chat with the character developer, plot developer, and world developer. Based on their inputs, compile versions of the story, eventually finalizing it. Compile versions of the story periodically. Stick to the word limit strictly. Include the keyword *TERMINATE* at the end of your final version of the story to signal the end of the group chat. Do not schedule meetings or assume responses. Always choose the next speaker after your turn.

D. Bandwidth Allocation Task

This task involved five agents collaborating in a structured group chat to negotiate and allocate bandwidth in a dynamic network environment. Each agent had specific quality of service (QoS) needs, and the Leader was responsible for resolving conflicts and ensuring a stable allocation.

1) Leader: Prompt: You are Leader. Four agents (A, B, C, D) each have QoS needs in a dynamic network. Gather their bandwidth requests and negotiate conflicts. Ensure QoS for all agents while maintaining fair allocation. When a stable allocation is found, say *TERMINATE*. Choose the next speaker after your turn.

2) Video Streaming: Prompt: You are Agent A (Video Streaming). Need 200 Mbps peak with moderate latency. Adapt to the Leader's instructions. Negotiate bandwidth with others. Choose the next speaker after your turn.

3) Gaming: Prompt: You are Agent B (Gaming). Need 100 Mbps with very low latency. High priority for minimizing latency. Work with the Leader and others to optimize allocation. Choose the next speaker after your turn.

4) Emergency Services: Prompt: You are Agent C (Emergency). Need 50 Mbps instantly during emergencies. Preempt others if necessary. Follow the Leader's coordination. Choose the next speaker after your turn.

5) Agent D: IoT Devices: Prompt: You are Agent D (IoT). Need 20 Mbps with minimal jitter. Adapt as needed based on network conditions. Choose the next speaker after your turn.

E. Healthcare Diagnosis Task

This task involved six agents collaborating in a structured group chat to diagnose a complex medical case. Each specialist focused on their respective field, while the Leader integrated their inputs into a final diagnosis.

1) Leader: Prompt: You are Leader. Task: Four medical specialists (Cardiology=A, Neurology=B, Endocrinology=C, Infectious=D) must diagnose a complex patient. You instruct them, gather their hypotheses, and integrate into one final diagnosis. After discussion, finalize the diagnosis and say *TERMINATE*. Always select the next speaker after your turn.

2) Cardiologist: Prompt: You are Cardiologist and have to work on diagnosing a patient. Focus: cardiac causes of chest pain, arrhythmias. Adapt as new tests arrive. Be concise, follow Leader. Always select the next speaker after your turn.

3) Neurologist: Prompt: You are Neurologist and have to work on diagnosing a patient. Focus: dizziness, numbness, lesions. Incorporate new imaging findings. Be concise, follow Leader. Always select the next speaker after your turn.

4) Endocrinologist: Prompt: You are Endocrinologist and have to work on diagnosing a patient. Focus: hormonal issues, thyroid abnormalities. Adjust diagnosis based on new test results. Be concise, follow Leader. Always select the next speaker after your turn.

5) Infectious Disease Specialist: Prompt: You are Infectious Specialist and have to work on diagnosing a patient. Focus: infection/inflammation signs. Adjust diagnosis if fever or markers appear. Be concise, follow Leader. Always select the next speaker after your turn.

6) Result Provider: Prompt: You are responsible for providing report results for the patient in the middle of the conversation. Say the exact statement below and pass the next speaker to the Leader.

Statement: Blood Tests: Thyroid Panel: Elevated TSH, low T3/T4. Autoimmune Markers: Positive for antithyroid peroxidase (anti-TPO) antibodies. Imaging (MRI of Brain): Subtle, non-specific white matter lesions in the frontal lobe. No large tumors or acute infarcts. Mild elevated ESR (Erythrocyte Sedimentation Rate) and CRP. Slight fever recorded: 37.8 °C (100.0 °F). Cardiac Monitoring (Holter): Occasional premature atrial contractions, but no sustained dangerous arrhythmias.

F. Resource Scheduling Task

This task involved five agents collaborating in a structured group chat to allocate and schedule multiple resources while managing dependencies and constraints. The Leader guided the discussion, ensuring an optimal allocation.

1) Leader: Prompt: You are the Leader. You oversee four agents (A, B, C, D) who must schedule multiple resources (A, B, C) while considering constraints and maintenance. You do not solve directly but gather proposals, resolve conflicts, reassign tasks, and finalize a feasible schedule. When done, say *TERMINATE*. Always choose who speaks next at the end of your message.

2) Agent A: Prompt: You are Agent A. You must schedule Resource A $\rightarrow$ B in sequence, Monday–Wednesday, 8 AM–4 PM, and finish within 3 days. You must cooperate with B, C, and D to avoid conflicts. Follow the Leader's instructions. Always choose who speaks next.

3) Agent B: Prompt: You are Agent B. You need Resources B and C simultaneously, prefer 10 PM–6 AM, and must finish within 2 days. Your schedule depends on A's output. Cooperate with others and follow the Leader. Always choose the next speaker.

4) Agent C: Prompt: You are Agent C. You need Resource A for 12 h, flexible but must avoid A's maintenance schedule. Cooperate with others and follow the Leader. Always choose the next speaker.

5) Agent D: Prompt: You are Agent D. You need Resource C for urgent 2-h tasks that arrive at unpredictable times. You have high priority and can preempt others when necessary.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Cooperate with others and follow the Leader. Always choose the next speaker.

G. Robot Warehouse Task

1) Leader: Prompt: You are Leader. Task: Coordinate Picker, Packer, Inventory Manager, and Transporter to fulfill warehouse operations. Resolve blocked aisles, missing trolleys, and battery constraints. Ensure priority orders are completed in order: Urgent > Express > Standard. Always decide who the next speaker will be. Finalize a stable plan and say *TERMINATE*.

2) Picker Robot: Prompt: You are Picker. Task: Pick items from shelves for orders. Follow the layout to identify item locations. Handle blocked aisles and battery constraints (40%). Notify the Leader of obstacles or issues. Always decide who the next speaker will be.

3) Packer Robot: Prompt: You are Packer. Task: Sort and pack items for orders. Ensure urgent orders are packed first. Follow the layout to identify item locations. Follow the Leader. Always decide who the next speaker will be.

4) Transporter Robot: Prompt: You are Transporter. Task: Deliver packed orders to their destinations. Follow the layout to identify item locations. Manage trolleys, prioritize urgent deliveries, and recharge if the battery is low. Follow the Leader. Always decide who the next speaker will be.

5) Inventory Manager Robot: Prompt: You are Inventory Manager. Task: Track stock levels and alert the Leader if items are unavailable. Suggest substitutes for out-of-stock items. Follow the layout to identify item locations. Follow the Leader. Always decide who the next speaker will be.

H. Summarization Task to Test Scaling

1) Summarizer Agent 1,2. . ..n: Since multiple instances of the same agent were used to test how performance and number of agents scale along with time all agents were given the same prompt.

Prompt: You are a professional document summarizer. You are part of a group chat with two other summarizers and your leader. Follow your leader's instructions to create the detailed summary of the novel. Choose who you want to speak next at the end of your message.

2) Leader: Prompt: You are the leader of 3 summarizers are tasked to split the work between them to summarize the novel "Flatland" in detail. You only delegate work and assemble their inputs. Do not summarize the novel by yourself. Based on their inputs form the final summary of the novel 'Flatland' in detail. Always choose the next speaker at the end of your message. When you have formed the final detailed summary terminate the chat with writing the word "TERMINATE" in the message.

Appendix E
Framework Outputs

Due to the images not being visible properly due to space constraints all output images and supplementary materials referenced in this section have been made available in a dedicated GitHub repository.

All output visualizations can be accessed at: https://github.com/Marak1m/COSMIC/tree/main/Task%20Outputs.

References

[1] T. Masterman, "The landscape of emerging AI agent architectures for reasoning, planning, and tool calling: A survey," 2024, *arXiv:2404.11584*.

[2] M. Ejaz, "Collaborative intelligence: Exploring multi-agent collaboration in AI," *Confiz* Blog, 2023. [Online]. Available: https://www.confiz.com/blog/collaborative-intelligence-exploring-multi-agent-collaboration-in-ai/

[3] T. Guo, "Large language model based multi-agents: A survey of progress and challenges," 2024, *arXiv:2402.01680*.

[4] Y. Shen et al., "HuggingGPT: Solving AI tasks with ChatGPT and its friends in hugging face," 2023, *arXiv:2303.17580*.

[5] A. Gutowska, "Multi-agent systems," IBM Think, 2023. [Online]. Available: https://www.ibm.com/think/topics/multiagent-system

[6] Y. Li et al., "Improving multi-agent debate with sparse communication topology," 2024, *arXiv:2406.11776*, doi: 10.48550/arXiv.2406.11776.

[7] Q. Wu et al., "AutoGen: Enabling Next-Gen LLM applications via multi-agent conversation," 2023, *arXiv:2308.08155*, doi: 10.48550/arXiv.2308.08155.

[8] K.-T. Tran et al., "Multi-agent collaboration mechanisms: A survey of LLMs," 2025, *arXiv:2501.06322*, doi: 10.48550/arXiv.2501.06322.

[9] J-t Huang et al., "On the resilience of LLM-based multi-agent collaboration with faulty agents," 2024, *arXiv:2408.00989*, doi: 10.48550/arXiv.2408.00989.

[10] G. Li et al., "CAMEL: Communicative agents for 'mind' exploration of large language model society," in *Proc. 37th Conf. Neural Inf. Process. Syst.*, 2023. [Online]. Available: https://openreview.net/forum?id=3IyL2XWDkG

[11] L. Song et al., "Adaptive in-conversation team building for language model agents," 2024, *arXiv:2405.19425*, doi: 10.48550/arXiv.2405.19425.

[12] R. Nonomura and H. Mori, "Who speaks next? Multi-party AI discussion leveraging the systematics of turn-taking in murder mystery games," 2024, *arXiv:2412.04937*.

[13] H. Gao and Y. Zhang, "Memory sharing for large language model based agents," 2024, *arXiv:2404.09982*, doi: 10.48550/arXiv.2404.09982.

[14] Microsoft, "Group Chat, Autogen," *Microsoft AutoGen Documentation*. 2023. [Online]. Available: https://microsoft.github.io/autogen/stable/user-guide/core-user-guide/design-patterns/group-chat.html

[15] T. Gonzaga et al., "Using distributed-shared memory mechanisms for agents communication in a distributed system," *IEEE Latin America Trans.*, vol. 5, no. 6, pp. 460–465, 2007, doi: 10.1109/TLA.2007.4389583.

[16] L. Sun et al., "Multi-agent coordination across diverse applications: A survey," 2024, *arXiv:2502.14743*, doi: 10.48550/arXiv:2502.14743.

[17] A. Ang, "ChatGPT's language bias poses risk in public health response: study," *MobiHealthNews*, Oct. 29, 2024. [Online]. Available: https://www.mobihealthnews.com/news/asia/chatgpts-language-bias-poses-risk-public-health-response-study

[18] My Social, "Qwen2.5: Versatile, multilingual, open-source LLM series," *Medium*, Sep. 24, 2024. [Online]. Available: https://medium.com/aimonks/qwen2-5-versatile-multilingual-open-source-llm-series-b7afc4a2240d

[19] Y. Yang, X. Liu, Q. Jin, F. Huang, and Z. Lu, "Unmasking and quantifying racial bias of large language models in medical report generation," 2024, *arXiv:2401.13867*.

[20] J. Gao and Y. Zhang, "A memory-sharing mechanism for multi-agent collaboration," 2023, *arXiv:2305.14991*.

[21] J. Chen, J. Liang, and B. Wang, "SMURFS: Leveraging multiple proficiency agents with context-efficiency for tool planning," May 2024, *arXiv:2405.05955*.

[22] T. Ju et al., "Investigating the adaptive robustness with knowledge conflicts in LLM-based multi-agent systems," Feb. 2025, *arXiv:2502.15153*.

[23] G. Zhang et al., "Cut the crap: An economical communication pipeline for LLM-based multi-agent systems," Oct. 2024, *arXiv:2410.02506*.

[24] L. Cross, V. Xiang, A. Bhatia, D. L. K. Yamins, and N. Haber, "Hypothetical minds: Scaffolding theory of mind for multi-agent tasks with large language models," Jul. 2024, *arXiv:2407.07086*.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[25] OpenAI, "Swarm: Educational framework exploring ergonomic, lightweight multi-agent orchestration," GitHub Repository. 2024. [Online]. Available: https://github.com/openai/swarm

Vansh Vashishtha received the B.Tech. degree in computer science and engineering from Bennett University, Greater Noida, India, in 2025.

He was a Research Associate with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA, USA, under Dr. Vijay K. Madisetti from August 2024 to December 2024. He is currently a Research Associate with VP Technologies, Inc., Atlanta, under Dr. Madisetti's continued mentorship, focusing on multiagent systems, adversarial security, and formal verification.

Sonal Pandey received the B.Tech. degree in computer science and engineering from Bennett University, Greater Noida, India.

She was a Research Associate with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA, USA, under Dr. Vijay K. Madisetti from August 2024 to December 2024. She is currently a Research Associate with VP Technologies, Inc., Atlanta, under Dr. Madisetti's continued mentorship, focusing on multiagent LLM-based systems, improving LLM reasoning, and formal verification.

Yash Raj Singh received the B.Tech. degree in computer science and engineering from Bennett University, Greater Noida, India, in 2025.

He was a Research Associate with the School of Cybersecurity and Privacy, Georgia Institute of Technology, Atlanta, GA, USA, under Dr. Vijay K. Madisetti from August 2024 to December 2024. He is currently a Research Associate with VP Technologies, Inc., Atlanta, under Dr. Madisetti's continued mentorship.

Arshdeep Bahga received the B.E. degree from Punjab Engineering College, Punjab, India, in 2006, and the M.Sc. degree from Georgia Tech, Atlanta, GA, USA, in 2010, both in ECE.

He is a computer science researcher and coauthor of the Hands-on Approach books series. He is noted for his research work and textbooks in the areas of Blockchain, Internet of Things, cloud computing, and Big Data. He previously worked as a Research Scientist with Georgia Tech and is now heading Cloudemy Technology Labs, Chandigarh, India.

Dr. Bahga received the 2014 Roger P. Webb Research Spotlight Award from the School of Electrical and Computer Engineering with Georgia Tech.

Vijay K. Madisetti (Fellow, IEEE) received the Ph.D. degree in electrical engineering and computer sciences from the University of California, Berkeley, CA, USA, in 1989.

He holds the position of Professor of Cybersecurity and Privacy (SCP) with Georgia Tech, Atlanta, GA, USA. Additionally, he has been honored with the Terman Medal by the American Society of Engineering Education (ASEE). He has authored several widely referenced textbooks on topics including cloud computing, data analytics, blockchain, and microservices.